

UNIVERSIDAD DE ORIENTE

**FACULTAD DE INGENIERÍA EN TELECOMUNICACIONES,
INFORMÁTICA Y BIOMÉDICA**

DEPARTAMENTO DE INGENIERÍA INFORMÁTICA



TRABAJO DE DIPLOMA

EN OPCIÓN DEL TÍTULO DE INGENIERO EN INFORMÁTICA

**TÍTULO: SISTEMA DE GESTIÓN DE ASOCIADOS PARA COMITÉ
PROVINCIAL DE LA UNEAC EN SANTIAGO DE CUBA.**

AUTOR: HIRSEL PAREDES VEGA

**TUTORES: ING. CARLOS ALBERTO BALBUENA PEÑA
MSc. HERSON TISSERT PÉREZ**

**SANTIAGO DE CUBA, 2025
AÑO 66 DE LA REVOLUCIÓN**

PENSAMIENTO

“Nunca se cambian las cosas luchando contra la realidad existente. Para cambiar algo, construye un nuevo modelo que haga obsoleto el modelo existente.”

Richard Buckminster Fuller

DEDICATORIA

A mi madre, pilar fundamental de mi vida. A mi esposa Zolanch García Melian, por su sacrificio incansable y apoyo constante durante este largo camino. A mis hijos, que son la razón más grande de mi esfuerzo y la inspiración que me impulsa a seguir adelante.

AGRADECIMIENTOS

A Dios, por ser mi guía constante y sostén en cada paso de este camino.

A mi madre, por estar presente en los momentos más cruciales de mi vida, brindándome apoyo incondicional y amor infinito.

A mi esposa, por su compañía incansable, por su paciencia y por ser mi fuerza silenciosa en los días más difíciles.

A mis hijos, por ser la inspiración que me impulsa a seguir adelante y el motivo más grande de mi esfuerzo.

A mis amistades que son como una familia, por su apoyo sincero, siempre presentes en los momentos de mayor necesidad.

A mis tutores, Carlos Alberto Balbuena Peña y Herson Tissert Pérez, por su dedicación, tiempo y apoyo sin límites, que hicieron posible transformar una idea en un proyecto sólido.

A mis compañeros de trabajo, por su confianza y colaboración en cada etapa de este proyecto, aportando siempre con compromiso y entusiasmo.

A mis compañeros de la carrera de Informática CPE, por los momentos compartidos desde el inicio de la carrera, que se convirtieron en recuerdos imborrables y aprendizajes que trascienden lo académico.

A todos los profesores del Departamento de Informática, por las enseñanzas que marcaron mi formación y sembraron en mí la pasión por la ingeniería.

Y finalmente, a todas las personas que de una forma u otra contribuyeron a mi crecimiento personal y profesional, porque cada gesto, cada palabra y cada apoyo han sido parte esencial de este logro.

Muchas gracias.

RESUMEN

La Unión de Escritores y Artistas de Cuba (UNEAC) enfrenta desafíos en la gestión de sus asociados debido a procesos manuales, descentralizados y propensos a errores, lo que limita la eficiencia operativa y la calidad de la información. Para resolver esta problemática, se desarrolló un sistema de gestión de asociados implementado como una aplicación web que centraliza y automatiza el registro, actualización, control de cotizaciones, organización de eventos y comunicación con los miembros del Comité Provincial de la UNEAC en Santiago de Cuba.

El desarrollo se basó en la metodología ágil Programación Extrema (XP), utilizando Python como lenguaje de programación, el framework Django, el entorno Visual Studio Code y el gestor de base de datos PostgreSQL. A partir de la interacción continua con los usuarios, se definieron historias de usuario que guiaron la implementación de funcionalidades clave, tales como la administración del perfil de los asociados, el seguimiento de pagos, la planificación de actividades culturales y el envío automatizado de notificaciones.

La solución propuesta mejora significativamente la trazabilidad de la información, reduce errores manuales, agiliza la generación de reportes y facilita la toma de decisiones mediante datos actualizados y confiables. Además, garantiza mayor eficacia, eficiencia y calidad en la gestión administrativa de la organización, alineándose con las necesidades reales del proceso y con los principios de modernización tecnológica impulsados en el sector cultural cubano.

Palabras clave: gestión de asociados, aplicación web, Programación Extrema, Django, UNEAC.

TITLE: ASSOCIATE MANAGEMENT SYSTEM FOR THE PROVINCIAL COMMITTEE OF THE UNEAC IN SANTIAGO DE CUBA.

ABSTRACT

The Union of Writers and Artists of Cuba (UNEAC) faces challenges in managing its members due to manual, decentralized, and error-prone processes, which limit operational efficiency and information quality. To address this issue, a member management system was developed as a web application that centralizes and automates membership registration, updates, membership dues control, event organization, and communication with members of the UNEAC Provincial Committee in Santiago de Cuba.

The development was based on the agile Extreme Programming (XP) methodology, using Python as the programming language, the Django framework, the Visual Studio Code environment, and the PostgreSQL database management system. Through ongoing interaction with users, user stories were defined that guided the implementation of key features, such as member profile management, payment tracking, cultural activity planning, and automated notification sending.

The proposed solution significantly improves information traceability, reduces manual errors, streamlines report generation, and facilitates decision-making through up-to-date and reliable data. It also ensures greater effectiveness, efficiency, and quality in the organization's administrative management, aligning with the real needs of the process and the principles of technological modernization promoted in the Cuban cultural sector.

Keywords: member management, web application, Extreme Programming, Django, UNEAC.

ÍNDICE

Introducción	1
Estructura de la Tesis	6
Capítulo 1 Fundamentación Teórica.....	7
1.1 Introducción	7
1.2 Estado del Arte	7
1.2.1 A escala internacional	8
1.2.2 A escala Nacional.....	9
1.3 Conceptos generales asociados a la investigación.....	10
1.3.1 Sistema Informático.....	10
1.3.2 Aplicación Web.....	10
1.3.3 Gestión	11
1.3.4 Sistema de Gestión	11
1.3.5 Organización	11
1.3.6 Asociación	11
1.3.7 Filial.....	11
1.3.8 Asociado / Miembro.....	12
1.3.9 Membresía	12
1.4 Herramientas y tecnologías utilizadas	12
1.4.1 Django (5.2.7).....	12
1.4.2 Visual Studio Code (1.105.0).....	12
1.4.3 PostgreSQL (Versión 14.3)	13
1.4.4 PgAdmin4 (Versión 8.9)	13
1.4.5 Visual Paradigm (Versión 8.0).....	14
1.4.6 Waitress (Versión 3.0.2).....	14
1.5 Metodología de desarrollo	15

1.5.1 Programación Extrema (XP).....	15
1.6 Lenguajes empleados.....	16
1.6.1 Lenguajes del lado del servidor (Backend).....	17
Python (versión 3.14.0)	17
SQL (Structured Query Language).....	17
1.6.2 Lenguajes del lado del cliente (Frontend)	17
HTML 5	18
JavaScript	18
CSS.....	18
1.6.3 Lenguaje de modelado unificado (UML).....	18
1.7 Conclusiones del capítulo	20
Capítulo II Descripción de la solución propuesta.....	21
2.1 Introducción	21
2.2 Descripción del proceso de gestión de los asociados.....	21
2.3 Propuesta del sistema	23
2.4 Roles y responsabilidades.	24
2.5 Usuarios del sistema.....	26
2.6 Funcionalidades del sistema.....	27
2.7 Historias de Usuario.....	28
2.8 Tareas de Ingeniería.....	30
2.9 Historias Técnicas.....	38
2.9.1 Requisitos de apariencia o interfaz externa.....	38
2.9.2 Requisitos de usabilidad.....	38
2.9.3 Requisitos de confidencialidad	38
2.9.4 Requisitos de fiabilidad	39
2.9.5 Requisitos de Disponibilidad	39
2.9.6 Requisitos de Software	39
2.9.7 Requisitos de Hardware	39

2.9.8 Requisitos de Seguridad	39
2.9.9 Requisitos de Rendimiento.....	40
2.10 Diagrama de Clases	40
2.11 Diagrama de la Base de Datos	41
2.12 Normalización	43
2.13 Tarjetas CRC	44
2.14 Diagrama de Despliegue	45
2.15 Patrón de Arquitectura	47
2.16 Conclusiones del capítulo	48
Capítulo III Implementación y pruebas	49
3.1 Introducción.....	49
3.2 ¿Cómo se aplica la filosofía de Django en la implementación del sistema?	50
3.3 Estructura de archivos fundamentales en un proyecto Django	50
3.4 Implementación del Sistema	53
3.4.1 Conectando la Base de Datos	53
3.4.2 Creando los modelos	54
3.4.3 La Vista	56
3.4.4 Las Plantillas	57
3.4.5 Las URLs	58
3.5 Paquetes Utilizados	59
3.6 Manejo de la seguridad del sistema.....	60
3.6.1 Implementación del Sistema de Autorización Personalizado.....	61
3.7 Pruebas realizadas al sistema	62
3.8 Validación.....	66
3.8.1 Validación en formularios	66
3.8.2 Validación en modelos	66
3.8.3 Reglas de Negocio Específicas de la UNEAC.....	67

3.9 Conclusiones del Capítulo	68
Conclusiones Generales	69
Recomendaciones.....	70
Referencias Bibliográficas	71
Anexos	74

ÍNDICE DE FIGURAS

Figura 1. 1 Fases de la Metodología XP	15
Figura 2. 1 Diagrama de Clases del diseño.....	40
Figura 2. 2 Figura Diagrama de la Base de Datos	43
Figura 2. 3 Diagrama de despliegue.....	46
Figura 2. 4 Patrón de Arquitectura MTV	47
Figura A- 1 Autenticación de usuarios.....	77
Figura A- 2 Página principal Dashboard.....	77
Figura A- 3 Página de Gestión de Asociados.....	78
Figura A- 4 Formulario de Gestión de Asociados	78
Figura A- 5 Página de Gestión de Eventos y actividades.....	79
Figura A- 6 Página de Gestión de Cotizaciones.....	79
Figura A- 7 Página de Reportes de Asociados.....	80
Figura A- 8 Página Gráfica de Distribución de Asociados por filial.....	80
Figura A- 9 Página Gráfica de Distribución de Géneros de los Asociados por filial	81
Figura A- 10 Página Gráfica de Distribución de Asociados por grupos de edad	81
Figura A- 11 Página Gráfica de Distribución de Asociados por género y edades ..	82

ÍNDICE DE TABLAS

Tabla 2 1 Estimación de mejora de tiempos de gestión	24
Tabla 2 2 Roles y Responsabilidades	25
Tabla 2 3 Usuarios del sistema	26
Tabla 2 4 Historia de Usuario: “Iniciar Sesión”	28
Tabla 2 5 Historia de Usuario: “Gestionar Asociados”	28
Tabla 2 6 Historia de Usuario: “Gestionar Cotizaciones”	29
Tabla 2 7 Historia de Usuario: “Gestionar Usuarios del Sistema”	29
Tabla 2 8 Tarea de Ingeniería: “Diseño del formulario de inicio de sesión”	30
Tabla 2 9 Tarea de Ingeniería: “Implementación de la lógica de autenticación”	30
Tabla 2 10 Tarea de Ingeniería: “Configuración de roles y permisos”	31
Tabla 2 11 Tarea de Ingeniería: “Validaciones y mensajes de error”	31
Tabla 2 12 Tarea de Ingeniería: “Seguridad en almacenamiento de contraseñas”	32
Tabla 2 13 Tarea de Ingeniería: “Implementación del modelo Django para asociados”	32
Tabla 2 14 Tarea de Ingeniería: “Desarrollo de vistas CRUD para asociados”	32
Tabla 2 15 Tarea de Ingeniería: “Diseño de formularios y validaciones de datos”	33
Tabla 2 16 Tarea de Ingeniería: “Configuración de permisos por rol”	33
Tabla 2 17 Tarea de Ingeniería: “Registro de auditoría para cambios en asociados”	34
Tabla 2 18 Tarea de Ingeniería: “Implementación del modelo Django para cotizaciones”	34
Tabla 2 19 Tarea de Ingeniería: “Desarrollo de vistas CRUD para cotizaciones”	34
Tabla 2 20 Tarea de Ingeniería: “Diseño de formularios y validaciones de datos”	35
Tabla 2 21 Tarea de Ingeniería: “Configuración de permisos por rol”	35
Tabla 2 22 Tarea de Ingeniería: “Registro de auditoría para operaciones de cotización”	36
Tabla 2 23 Tarea de Ingeniería: “Implementación del modelo Django para usuarios y roles”	36
Tabla 2 24 Tarea de Ingeniería: “Desarrollo de vistas CRUD para usuarios”	36
Tabla 2 25 Tarea de Ingeniería: “Asignación de roles y permisos”	37
Tabla 2 26 Tarea de Ingeniería: “Validaciones de seguridad”	37

Tabla 2 27 Tarea de Ingeniería: “Registro de auditoría para cambios en usuarios”	38
Tabla 2 28 Tarjeta CRC.....	44
Tabla 2 29 Tarjeta CRC.....	45
Tabla 2 30 Tarjeta CRC.....	45
Tabla 2 31 Tarjeta CRC.....	45
Tabla 3. 1 Prueba de Aceptación: Autenticar Usuarios	63
Tabla 3. 2 Prueba de Aceptación: Registrar Asociado	64
Tabla 3. 3 Prueba de Aceptación: Registrar Cotización	64
Tabla 3. 4 Prueba de Aceptación: Crear Usuario del Sistema	65
Tabla A- 1 Prueba unitaria: Validación de credenciales correctas	74
Tabla A- 2 Prueba unitaria: Validación de credenciales incorrectas.....	74
Tabla A- 3 Prueba de Funcionalidad 1: Inicio de sesión exitoso	75
Tabla A- 4 Prueba de Funcionalidad 2: Inicio de sesión fallido	76

INTRODUCCIÓN

El acelerado desarrollo de las Tecnologías de la Información y las Comunicaciones (TIC) ha transformado profundamente la manera en que las organizaciones operan y gestionan sus procesos. En este contexto, los Sistemas de Gestión han emergido como herramientas fundamentales, integrando componentes organizativos, técnicos y tecnológicos para recopilar, almacenar, procesar y distribuir información de manera eficiente. Estos sistemas representan la evolución natural de una necesidad humana constante: la de organizar comunidades y asignar roles para el bienestar colectivo. Desde las primeras agrupaciones sociales hasta las estructuras jerárquicas modernas, la gestión de miembros ha sido un pilar para el funcionamiento de cualquier colectividad.

Los sistemas de gestión de asociados o Member Management Systems (MMS) constituyen un ejemplo paradigmático de esta evolución. Diseñados para abordar las necesidades específicas de las organizaciones, permiten la administración integral de la información de los miembros, facilitan la comunicación, automatizan la gestión de pagos y cotizaciones, coordinan eventos y generan informes detallados. Estas capacidades optimizan las operaciones, mejoran la interacción con los asociados y proveen una base sólida para la toma de decisiones. [1]

La Unión de Escritores y Artistas de Cuba (UNEAC), como organización social y cultural de relevancia nacional, no es ajena a estos desafíos. Agrupa a creadores de diversas disciplinas a través de asociaciones y comités provinciales, con el loable objetivo de estimular, proteger y difundir la creación artística e intelectual cubana. Uno de sus procesos fundamentales es la gestión de sus asociados, el cual, pese a su crítica importancia, se sustenta en métodos predominantemente manuales.

Actualmente, la gestión de los asociados en el Comité Provincial de la UNEAC en Santiago de Cuba se caracteriza por una dependencia significativa de planillas físicas, documentos digitales no integrados (como hojas de cálculo Excel y procesadores de texto) y un flujo de información descentralizado. La actualización de los datos de los miembros, el registro de pagos de cotizaciones y la organización de eventos requieren de una considerable inversión de tiempo y esfuerzo por parte del personal administrativo. Procedimientos como la invitación a actividades, que se realiza mediante contactos telefónicos individuales, o la conciliación de pagos, que

implica el manejo de comprobantes físicos por triplicado, son ejemplos de la fragmentación operativa. Esta realidad conlleva inherentemente a la duplicidad de información, inconsistencia en los datos, dificultades para generar reportes unificados y riesgos para la integridad y seguridad de la documentación.

En consecuencia, la falta de una plataforma unificada que centralice y automatice estos procesos se identifica como la limitante principal que obstaculiza la eficacia, eficiencia y escalabilidad de la gestión de asociados. Esto evidencia la necesidad imperante de desarrollar una solución informatizada que optimice estos flujos de trabajo, libere capacidades del personal para tareas de mayor valor y, con ello, fortalezca el cumplimiento de los elevados objetivos culturales y sociales de la UNEAC.

A raíz de la situación problemática antes descrita se formula el siguiente **problema de investigación**; Las limitaciones en el proceso de gestión de asociados del Comité Provincial de la UNEAC en Santiago de Cuba, caracterizadas por la falta de automatización, ineficiencias en el manejo de la información y administración descentralizada de datos.

Identificándose como el **objeto de estudio** de la investigación los procesos de gestión de asociados y como **campo de acción** de la investigación la gestión de asociados pertenecientes al Comité Provincial de la Uneac en Santiago de Cuba.

Se propone como **Objetivo General**: Diseñar e implementar una solución tecnológica que optimice los procesos de gestión de asociados del Comité Provincial de la UNEAC en Santiago de Cuba, atendiendo a las limitaciones actuales en automatización, manejo de la información y administración de datos.

Para darle cumplimiento al objetivo planteado se definieron los siguientes **Objetivos específicos**:

1. Caracterizar el proceso actual de gestión de asociados en el Comité Provincial de la UNEAC en Santiago de Cuba, incluyendo el análisis de los métodos manuales y herramientas informáticas empleadas (planillas físicas, hojas de cálculo y documentos dispersos).
2. Analizar el estado del arte en sistemas de gestión de asociados a nivel nacional e internacional, identificando soluciones existentes, sus limitaciones

en el contexto cubano y justificando la necesidad de una propuesta adaptada al sector cultural.

3. Identificar y evaluar las herramientas tecnológicas necesarias para el diseño e implementación del sistema de gestión de asociados que optimice la administración, actualización y trazabilidad de la información de los miembros.
4. Definir y documentar la metodología de desarrollo de software que guiará la implementación del sistema, justificando la selección de Extreme Programming (XP) por su adecuación al contexto institucional y a los requisitos cambiantes identificados.
5. Diseñar e implementar el sistema de gestión de asociados como aplicación web en el Comité Provincial de la UNEAC, asegurando su integración con los procesos administrativos existentes y capacitando al personal para su uso efectivo.
6. Validar la solución propuesta mediante pruebas de calidad de software (unitarias y de funcionalidad) y pruebas de aceptación realizadas por usuarios finales y directivos del Comité Provincial de la UNEAC.
7. Desplegar el sistema en el entorno operativo del Comité Provincial de la UNEAC en Santiago de Cuba, asegurando su correcto funcionamiento, disponibilidad continua en la red institucional y soporte técnico para futuras mejoras y extensiones.

La investigación plantea la siguiente **Hipótesis** a defender: Si se implementa una solución tecnológica orientada a la automatización de los procesos de gestión de asociados en el Comité Provincial de la UNEAC en Santiago de Cuba, entonces se incrementará significativamente la eficiencia operativa, evidenciada en la reducción del tiempo de actualización y consulta de información, la disminución de errores manuales y la mejora en la generación de reportes estructurados para la toma de decisiones institucionales.

Para la realización de la investigación fueron empleados los siguientes **Métodos Científicos**:

Método analítico–sintético: Se aplicó en el estudio de la bibliografía y documentación asociada al objeto de estudio, identificando los elementos más relevantes para descomponer la problemática, abstraer sus causas y,

posteriormente, integrar conclusiones que sustentan la necesidad de la investigación y orientan el diseño de la solución.

Método histórico–lógico: Se utilizó para examinar la evolución de sistemas y soluciones informáticas implementadas por organizaciones del país y del ámbito internacional, así como para analizar la problemática existente en el proceso de gestión de asociados. Este análisis permitió fundamentar, con base en antecedentes y regularidades, la definición de requerimientos y la orientación de la propuesta de solución.

Método sistémico–estructural–funcional: Se empleó para modelar el proceso de gestión de asociados desde una perspectiva integral, identificando componentes, relaciones y funciones esenciales del sistema. Este enfoque facilitó la delimitación de módulos, flujos de información y responsabilidades, asegurando la coherencia interna de la arquitectura propuesta y su alineación con los objetivos institucionales.

Técnicas empleadas (métodos empíricos)

Observación: Permitió comprender el proceso que se desea informatizar, identificando tareas, actores, entradas, salidas y restricciones del flujo de trabajo en la gestión de asociados.

Entrevistas: Se realizaron entrevistas a trabajadores de la Secretaría de Asociaciones y a personal de departamentos administrativos para validar necesidades, levantar requerimientos de nuevas funcionalidades y detectar oportunidades de mejora en el proceso existente.

Se analizaron metodologías ágiles como Scrum, Kanban, XP y Crystal, dado que todas ellas promueven flexibilidad en la planificación y adaptación continua. Estos marcos facilitan la evolución de los requerimientos, asegurando entregas incrementales con retroalimentación frecuente de los actores clave y contribuyen a mitigar riesgos mediante iteraciones cortas, manteniendo trazabilidad entre necesidades, historias de usuario y componentes implementados.

Dentro de este ecosistema ágil, se optó por Extreme Programming (XP) debido a su énfasis en la calidad técnica y la adaptación continua: pruebas automatizadas, integración frecuente, diseño simple y refactorización sistemática. Estas prácticas permiten reducir la deuda técnica, mejorar la difusión del código y acortar el ciclo

entre el descubrimiento de necesidades y la entrega funcional, aspectos críticos en contextos institucionales caracterizados por la validación directa de usuarios y la presencia de cambios normativos.

Aportes prácticos que el sistema brindará:

- 1- **Eficiencia en la gestión de los asociados:** Al automatizar el registro y la actualización de los asociados, se reduce significativamente el tiempo y el esfuerzo necesario para la realización de estas tareas, contribuyendo a la disminución de errores humanos y garantizar que la información de los asociados esté siempre actualizada y precisa.
- 2- **Gestión eficaz de las cotizaciones:** El sistema facilita el registro y la administración y trazabilidad de las cotizaciones de los miembros, lo que mejora en la precisión y la eficiencia de estos procesos.
- 3- **Organización de eventos y actividades:** Con un calendario de eventos, reuniones y actividades centralizado, al personal administrativo les permite tener una visión clara de los próximos eventos y puede planificar la realización de manera efectiva.
- 4- **Soporte para la toma de decisiones:** Al centralizar toda la información relacionada con los asociados y su membresía y además de tener la posibilidad de generación de reportes y graficas basadas en datos, el sistema permite facilitar la toma de decisiones basada en datos para los directivos de una manera más informada, asertiva y efectiva.
- 5- **Seguridad de la información:** Un sistema proporcionará medidas de seguridad para proteger la integridad de la información de los asociados y garantizar su privacidad.

Estructura de la Tesis

El presente trabajo de diploma se ha estructurado de la siguiente forma: Resumen, Introducción, tres capítulos, Conclusiones, Recomendaciones, Bibliografía y Anexos. El contenido de los capítulos se organiza de la siguiente manera:

Capítulo 1: **Fundamentación teórica.** En este capítulo se establecen los fundamentos teóricos que sustentan la investigación. Se presenta un estudio del estado del arte sobre sistemas de gestión de asociados, analizando soluciones existentes y tendencias tecnológicas. Asimismo, se detalla la selección y justificación de la metodología de desarrollo, las tecnologías, lenguajes y herramientas de desarrollo y los patrones de diseño y arquitectónicos empleados.

Capítulo 2: **Diseño y modelado del sistema.** Este capítulo contiene el análisis y diseño del sistema. Se inicia con la descripción y modelado de los procesos de negocio a informatizar. A partir de ello, se derivan y especifican los requisitos funcionales y no funcionales. Siguiendo los principios de XP, los requisitos se materializan en historias de usuario y sus correspondientes historias técnicas, las cuales guían el diseño. Se presenta el modelo de datos, la arquitectura del sistema, los patrones de diseño aplicados y el diseño detallado de las funcionalidades principales.

Capítulo 3: **Implementación y pruebas.** Este capítulo documenta la construcción y validación del sistema. Se detalla el proceso de implementación de los módulos y funcionalidades definidos, explicando cómo se cumplieron las historias de usuario. Se describen las pruebas realizadas, los escenarios evaluados y los resultados obtenidos.

CAPÍTULO 1 FUNDAMENTACIÓN TEÓRICA

1.1 Introducción

El hombre, por naturaleza, es un ser social que siempre ha buscado colaborar e interactuar con otros para realizar diversas tareas. A medida que estas sociedades crecían en tamaño y complejidad, surgió la necesidad de implementar sistemas para gestionar eficientemente estas asociaciones. Los sistemas de gestión, en su forma más básica, podrían haber comenzado como simples acuerdos o normas establecidas por el grupo. Con la evolución del hombre y las sociedades, los sistemas de gestión también han evolucionado, pasando de ser simples acuerdos sociales a sistemas complejos que incorporan principios de administración, economía y gestión de recursos, estrechamente relacionados con la tecnología. Hoy en día, los sistemas de gestión forman la columna vertebral de la teoría y práctica organizacional, agilizan las operaciones, automatizan tareas y centralizan datos para mejorar la eficacia y eficiencia en las organizaciones. [2] La relación entre los sistemas de gestión y las tecnologías es crucial, ya que las tecnologías sirven como herramientas para transformar los objetos de gestión en estados competitivos y garantizar un funcionamiento eficaz, con un enfoque en mejorar el desarrollo social y aumentar la eficiencia empresarial. [3]

El presente capítulo contiene la base teórica de este trabajo que está sustentada en los sistemas informáticos de gestión de asociados en organizaciones/asociaciones que sirven de punto de partida para la investigación. Se exponen los conceptos esenciales para comprender el problema planteado, así como la definición de las herramientas y tecnologías a utilizar durante el desarrollo de la propuesta de solución. Se realiza, además, un estudio de la metodología que servirá como guía para dar solución al problema planteado y asegurar que el proceso de gestión de asociados sea eficiente, eficaz y de alta calidad.

1.2 Estado del Arte

Para la creación de este sistema de gestión de asociados, se realizó el estudio del estado del arte, permitiendo así, la investigación y el estudio de sistemas análogos vinculados con el campo de acción, tanto en el ámbito internacional como nacional. De este modo, se adquirió una comprensión más profunda de estos sistemas, lo que

permitió elaborar un conjunto de posibles soluciones que satisfacen a los objetivos propuestos. Los hallazgos de esta investigación se presentan a continuación:

1.2.1 A escala internacional

Membee es un software de gestión de miembros desarrollado por la empresa estadounidense IRM Systems (United States). Este es un software de gestión de la adhesión basada en la nube diseñado para hacer la vida más fácil para los miembros, empleados y voluntarios. Este software añade una potente funcionalidad a tu sitio web a través de los “plug & play” widgets de Membee, mediante los cuales puedes implementar un nuevo registro de miembro, la renovación de miembros existentes, directorios de miembros, perfiles de los miembros, y el registro de eventos completo con el procesamiento de pagos. El precio de la licencia para el uso de este software es de \$55.00/mes/usuario. [4]

Member Manager es un Software de Gestión de Asociados (MMS) desarrollado por la empresa estadounidense del mismo nombre, Member Manager. Este software es una herramienta esencial para diversas organizaciones basadas en miembros, como asociaciones, organizaciones sin fines de lucro, clubes, cámaras de comercio y grupos comerciales. Su objetivo es ayudar a estas organizaciones a mejorar la eficiencia en áreas como la facturación de miembros, la comunicación y el compromiso. Ofrece una serie de características, entre las que se incluyen la comunicación con los miembros, una base de datos de miembros y un portal de miembros. Estas funcionalidades convierten a Member Manager en una herramienta en línea diseñada para optimizar la gestión de su club u organización a través de encuestas, redes sociales, calendarios y foros. El costo de la licencia para el uso de este software es de \$99.00 al año. [5]

MemberLeap es un software de gestión de miembros desarrollado por la empresa estadounidense Vieth Consulting. Este software incluye una amplia gama de características que pueden asistir en la administración de su asociación. Cuenta con un robusto sistema de base de datos de usuarios para rastrear las cuotas y comunicarse con los miembros; herramientas avanzadas de gestión de contenidos para mantener su sitio web actualizado de manera sencilla; un sistema para la convocatoria de propuestas, registro de eventos y exhibición de stands para gestionar presentadores, conferencias y ferias comerciales; herramientas de comunicación para mantener a los miembros comprometidos, y aprendizaje en línea

y seguimiento de Unidades de Educación Continua (CEU). El costo de este software es de \$40.00 por mes.[6]

MemberPress es un software de gestión de miembros desarrollado por la empresa estadounidense Caseproof. Se trata de un plugin de membresía para WordPress fácil de usar. Este potente software de membresía se instala de manera sencilla en su sitio y le permite comenzar a cobrar por el acceso de inmediato. A diferencia de muchas otras soluciones de software para miembros, MemberPress se integra completamente con su sitio web WordPress existente, es muy robusto y también fácil de usar. El costo de este software es de \$99.00 al año.[7]

Daxko Operations es un software de gestión de membresía desarrollado por la empresa estadounidense Daxko. Este software ayuda a administrar las operaciones cotidianas de su centro de manera segura. Es herramientas como el registro de programas, eficiencias de back-office como el acceso fácil a la información de los miembros, y procesos y datos estandarizados mediante cientos de YMCA, CCM y otras organizaciones de salud y bienestar en el paciente. Daxko Operations se considera el estándar de excelencia en la gestión de operaciones basadas en miembros. El costo de este software es de \$99.00 por mes. [8]

1.2.2 A escala Nacional

Mediante visitas realizadas a organizaciones en la provincia como la Unión Nacional de Arquitectos e Ingenieros de la Construcción de Cuba (UNAICC), la Asociación Hermanos Saíz (AHS), la Asociación Cubana de Limitados Físico-Motores (ACLIFIM), la Asociación Cubana de Técnicos Agrícolas y Forestales (ACTAF) y la Asociación de Combatientes de la Revolución Cubana (ACRC), se constató la ausencia de sistemas de software especializados para la gestión de asociados. En su lugar, estas organizaciones realizan el proceso de gestión mediante hojas de cálculo electrónicas (principalmente en Excel), bases de datos dispersas implementadas en gestores heterogéneos (Access, MySQL, PostgreSQL) e incluso, en algunos casos, mediante registros físicos en planillas de papel. Esto ocurre en cada caso a nivel provincial. No así en sus sedes a nivel nacional donde se utilizan sistemas de Planificación de Recursos Empresariales (ERP).

Tras un análisis exhaustivo de la literatura relacionada con los Sistemas de Gestión de Asociados (MMS, *Member Management System*, por sus siglas en inglés), se ha

determinado que las soluciones existentes en el ámbito internacional no se ajustan adecuadamente a circunstancias particulares de nuestro país, debido a que la adquisición de estas soluciones implicaría la compra de licencias, porque son de origen privativo, lo cual contradice la política de software libre de nuestro país. Por lo tanto, es evidente la necesidad de explorar alternativas que estén en línea con nuestras políticas y condiciones locales. En el contexto nacional, los sistemas de gestión de asociados utilizados a pesar de cumplir con estas políticas, son sistemas de Planificación de Recursos Empresariales (ERP) que planifican todos los recursos de la organización o empresa, lo que resulta un espectro muy amplio para lo que se desea implementar en el Comité Provincial de la Uneac. Además, la tecnología computacional con la que se trabaja en el centro tiene un alto nivel de obsolescencia, por lo que resulta uno de los aspectos más importantes al implementar una solución.

Por estas razones, proponemos la implementación de un sistema que pueda adaptarse a las necesidades actuales del cliente, aprovechando las principales ventajas observadas en algunas de las soluciones analizadas. Este sistema deberá ser capaz de gestionar exclusivamente a los miembros de la organización, cumplir con la política de software libre del país y demostrar su eficacia como sistema de gestión. De esta manera, podremos garantizar una solución que se ajuste a las necesidades del cliente y políticas específicas de la organización y del país.

1.3 Conceptos generales asociados a la investigación.

En esta sección, presentaremos conceptos de manera generalizada que facilitarán una mejor comprensión de la investigación en curso.

1.3.1 Sistema Informático

Se entiende como un conjunto de elementos interrelacionados que permiten el almacenamiento, procesamiento y recuperación de datos, que aprovecha las herramientas de la computación y la electrónica para llevar a cabo una serie compleja de procesos y operaciones. Estos elementos incluyen el hardware, el software y el usuario, quien requiere de la información procesada y tiene el control total de lo que sucede en el sistema. [9]

1.3.2 Aplicación Web

Una aplicación web (web-based application) es un tipo especial de aplicación cliente/servidor, donde tanto el cliente (el navegador, explorador o visualizador)

como el servidor (el servidor web) y el protocolo mediante el que se comunican (HTTP) están estandarizados y no han de ser creados por el programador de aplicaciones. [10]

1.3.3 Gestión

Acto y resultado de administrar un proceso, ya sea en un contexto empresarial o personal. Implica la implementación de una serie de políticas, procedimientos y acciones estratégicas con el propósito de alcanzar un objetivo específico. Requiere habilidades de planificación, organización, dirección y control para optimizar el uso de recursos y mejorar la eficiencia y efectividad del proceso en cuestión. [11]

1.3.4 Sistema de Gestión

Un Sistema de Gestión es un sistema informático compuesto por un conjunto interrelacionado de políticas, procesos, procedimientos y recursos utilizados para lograr los objetivos de una organización. Este sistema proporciona un marco estructurado para las operaciones eficientes de una organización lo que también permite la optimización a nivel de costos, calidad y tiempo facilitando las mejoras continuas. [12]

1.3.5 Organización

Una organización se entiende como una entidad legal constituida con el propósito de alcanzar objetivos específicos, caracterizada por una estructura jerárquica y funciones definidas. Según López de Pedro, las organizaciones coordinan personas y recursos para cumplir su misión.[13] De acuerdo con UniNotas, pueden clasificarse en públicas o privadas, con o sin fines de lucro.[14] Asimismo, Economía360 enfatiza que las organizaciones poseen personalidad jurídica que les permite realizar contratos y actividades económicas.[15]

1.3.6 Asociación

Grupo organizado de personas que se unen con un propósito común, que puede estar relacionado con su profesión, oficio, pasatiempo, o cualquier otro interés compartido. Las asociaciones suelen ser entidades legales y se rigen por estatutos que definen su propósito y funcionamiento.[16]

1.3.7 Filial

Entidad que forma parte de una organización o asociación más grande y que está bajo su control o influencia.[17]

1.3.8 Asociado / Miembro

Sujeto que pertenece a una organización, asociación o filial y tiene obligaciones y derechos dentro de la misma. [18]

1.3.9 Membresía

Estado o condición de ser un miembro o asociado de una organización o asociación, es la relación existente entre el individuo y la organización a la que pertenece. [19]

1.4 Herramientas y tecnologías utilizadas

1.4.1 Django (5.2.7)

Django es un framework de desarrollo web escrito en Python, diseñado para construir aplicaciones seguras, escalables y mantenibles. Se caracteriza por su enfoque en la rapidez de desarrollo y la reutilización de componentes, ofreciendo un conjunto completo de herramientas que incluyen ORM, sistema de plantillas, administración automática y soporte para pruebas. Su arquitectura sigue el patrón MTV (Model-Template-View), lo que facilita la separación de responsabilidades y la organización del código.[20] Además, es multiplataforma y compatible con Python 3.9 o superior, lo que lo convierte en una opción versátil tanto en proyectos académicos como profesionales. En el desarrollo de la aplicación se evaluaron alternativas como Flask y FastAPI, también utilizadas en entornos de desarrollo web. Sin embargo, se optó por Django debido a las siguientes ventajas:

- Estructura robusta y completa
- Sistema de administración integrado
- Seguridad y buenas prácticas incorporadas
- Amplia comunidad y documentación

La elección de Django para el desarrollo de la aplicación responde a criterios de robustez, seguridad, productividad y versatilidad en el desarrollo web profesional, lo que lo convirtió en la solución más adecuada frente a otras opciones como Flask o FastAPI en el contexto específico del proyecto.

1.4.2 Visual Studio Code (1.105.0)

Visual Studio Code es un editor de código fuente moderno, gratuito y de código abierto, desarrollado por Microsoft para ofrecer una experiencia de programación ágil, versátil y altamente personalizable. Gracias a su arquitectura ligera, su sistema de extensiones integradas, su depuración interactiva y su compatibilidad con

múltiples lenguajes y entornos, permite a desarrolladores de todos los niveles trabajar con eficiencia y precisión en proyectos locales o colaborativos. Ofrece funciones avanzadas como resaltado de sintaxis, control de versiones con Git, terminal integrada y asistencia inteligente mediante IntelliSense, lo que facilita la escritura de código limpio y estructurado.[21] Su comunidad activa, su integración fluida y su capacidad para adaptarse a flujos de trabajo complejos lo convierten en una herramienta ideal tanto para desarrolladores independientes como para equipos técnicos institucionales que buscan productividad, claridad visual y una base tecnológica confiable. Visual Studio Code se eligió por su flexibilidad y potencia en el desarrollo profesional multiplataforma.

1.4.3 PostgreSQL (Versión 14.3)

PostgreSQL es un sistema de administración de bases de datos relacionales (**RDBMS** por sus siglas en inglés, *Relational Database Management System*) de código abierto, robusto y extensible. Es conocido por sus funciones avanzadas, su cumplimiento de estándares y su compatibilidad con tipos de datos complejos, como matrices. Su énfasis en las propiedades de SQL y ACID garantiza la confiabilidad y la integridad de los datos. Además, su extensibilidad mediante varias extensiones permite la personalización según requisitos específicos. Destaca por sus mecanismos de indexación, optimización de consultas y control de simultaneidad, contribuyendo a un rendimiento excepcional. Las funciones principales de PostgreSQL incluyen la creación y mantenimiento de la base de datos, el control de accesos, la manipulación de datos según las necesidades del usuario y el mantenimiento de la integridad de los datos.[22] PostgreSQL se utilizó para el desarrollo de la solución propuesta debido a su robustez y fiabilidad como sistema de gestión de bases de datos relacional. Su compatibilidad con una amplia gama de tipos de datos, soporte para consultas complejas y alto rendimiento en operaciones de lectura y escritura lo hacen ideal para las necesidades del desarrollo de la solución. Además, su facilidad de integración con varios lenguajes de programación refuerza la elección de PostgreSQL para el desarrollo de la solución propuesta.

1.4.4 PgAdmin4 (Versión 8.9)

Es la principal herramienta de gestión de código abierto para PostgreSQL, la base de datos de código abierto más avanzada del mundo. PgAdmin4 está diseñado para satisfacer las necesidades tanto de los usuarios novatos como de los usuarios

experimentados de PostgreSQL, proporcionando una potente interfaz gráfica que simplifica la creación, el mantenimiento y el uso de objetos de bases de datos. Está disponible en más de una docena de lenguajes y para varios sistemas operativos, incluyendo Microsoft Windows, Linux, Mac OSX y Solaris.[23]. La elección de PgAdmin4 para la administración de la base de datos responde a criterios de integración, facilidad de uso y versatilidad

1.4.5 Visual Paradigm (Versión 8.0)

Visual Paradigm es una herramienta de Ingeniería de Software Asistida por Computación (CASE). Ha sido concebida para soportar el ciclo de vida completo del proceso de desarrollo del software a través de la representación de todo tipo de diagramas. Se caracteriza por tener disponibilidad en múltiples plataformas y se enfoca al negocio generando un software de mayor calidad. Esta herramienta permite aumentar la calidad del software, a través de la mejora de la productividad en el desarrollo y mantenimiento del software. También realiza la distribución automática de diagramas, así como la reorganización de las figuras y conectores de los diagramas UML.[24] La elección de Visual Paradigm para el modelado y documentación del sistema responde a criterios de estandarización, facilidad de uso y versatilidad, lo que lo convirtió en la solución más adecuada

1.4.6 Waitress (Versión 3.0.2)

Waitress es un servidor WSGI escrito en Python, diseñado para ejecutar aplicaciones web en entornos de producción. Se caracteriza por ser ligero, seguro y fácil de configurar, sin depender de librerías externas. Su arquitectura está optimizada para manejar múltiples solicitudes concurrentes de manera eficiente, soportando HTTP/1.0 y HTTP/1.1, y siendo compatible con Python 3.9 o superior. Además, es multiplataforma, funcionando tanto en sistemas Unix como en Windows, lo que lo convierte en una opción versátil para proyectos académicos y profesionales.[25] En el desarrollo de la aplicación se evaluaron alternativas como Gunicorn y uWSGI, ampliamente utilizadas en despliegues de aplicaciones Django. Sin embargo, se optó por Waitress debido a las siguientes ventajas:

- Compatibilidad con Windows
- Simplicidad de configuración
- Estabilidad y seguridad
- Ligereza

La elección de Waitress para el despliegue de la aplicación responde a criterios de compatibilidad, simplicidad y estabilidad, lo que lo convirtió en la solución más adecuada que otras opciones como Gunicorn o uWSGI en el contexto específico del proyecto.

1.5 Metodología de desarrollo

1.5.1 Programación Extrema (XP)

Es una Metodología ligera de desarrollo de aplicaciones que se basa en la simplicidad, la comunicación y la realimentación del código desarrollado. XP tiene como objetivos la satisfacción del cliente, potenciar el trabajo en grupo y minimizar el riesgo actuando sobre las variables del proyecto: costo, tiempo, calidad, alcance. Se caracteriza por ser una metodología basada en prueba y error para obtener un software que funcione realmente, fundamentada en principios. Está orientada hacia quien produce y usa el software (el cliente participa muy activamente). Reduce el coste del cambio en todas las etapas del ciclo de vida del sistema. También cuenta con un grupo pequeño y muy integrado (2-10 personas) para trabajar. [26] Esta metodología se divide en Cuatro fases como se muestra a continuación:



Figura 1. 1 Fases de la Metodología XP

Fase #1: Planificación

En esta fase, se plantean a grandes rasgos las Historias de Usuario (HU) que son de interés para la primera entrega del producto. Las HU son construidas con la información proporcionada por los clientes y el equipo de desarrollo comienza a familiarizarse con las herramientas, metodología y prácticas que serán usadas para realizar el proyecto. Las HU representan los requerimientos de software, y son descritas bajo el lenguaje del cliente.

Fase #2: Diseño

En esta fase se establece la prioridad de cada historia de usuario y los programadores establecen una estimación de esfuerzo necesario para cada una. El orden de las historias implementadas en las iteraciones será determinado por el cliente.

Fase #3: Codificación o Desarrollo

La programación aquí se hace en parejas en frente del mismo ordenador. De esta forma, nos aseguramos de que se realice un código más universal, con el que cualquier otro programador podría trabajar y entender. Y es que deber parecer que ha sido realizado por una única persona. Así se conseguirá una programación organizada y planificada.

Fase #4: Prueba

Se deben realizar pruebas automáticas continuamente. Al tratarse normalmente de proyectos a corto plazo, este testeo automatizado y constante es clave. Además, el propio cliente puede hacer pruebas, proponer nuevas pruebas e ir validando las pequeñas versiones.

1.6 Lenguajes empleados

En el desarrollo del sistema se utilizaron lenguajes de programación tanto del lado del servidor como del lado del cliente, cada uno con funciones específicas dentro de la arquitectura web. Esta separación permite una mejor organización de responsabilidades y optimiza el rendimiento de la aplicación.

1.6.1 Lenguajes del lado del servidor (Backend)

Python (versión 3.14.0)

Python es un lenguaje de programación de alto nivel, gratuito y de código abierto, diseñado para facilitar el desarrollo de soluciones claras, eficientes y escalables en una amplia variedad de contextos técnicos y científicos. Gracias a su sintaxis legible, su enfoque en la simplicidad estructural, su ecosistema de bibliotecas especializadas y su compatibilidad con múltiples paradigmas —como la programación orientada a objetos, funcional y procedimental— permite a desarrolladores, investigadores e instituciones construir desde scripts automatizados hasta sistemas complejos con rapidez y precisión. Ofrece herramientas robustas para análisis de datos, inteligencia artificial, desarrollo web, automatización, visualización y más, lo que lo convierte en un lenguaje versátil y adaptable a casi cualquier necesidad profesional.[27] Su comunidad global, su documentación extensa y su integración con Django lo consolidan como una opción estratégica para la productividad, claridad conceptual y una base tecnológica confiable. Python se eligió por su elegancia, potencia y flexibilidad en el desarrollo moderno e institucional.

SQL (Structured Query Language)

El lenguaje de consulta estructurado o SQL (Structured Query Language) es un lenguaje declarativo que permite realizar operaciones en bases de datos relacionales, como crear, procesar y almacenar datos. Destaca por su manejo del álgebra y el cálculo relacional, facilitando consultas para recuperar información y realizar cambios. SQL explota la flexibilidad y potencia de los sistemas relacionales, permitiendo desde simples consultas hasta transacciones complejas, y se utiliza para la manipulación y definición de datos mediante la creación y modificación de bases de datos, tablas e índices. [28]. En el desarrollo de la aplicación se empleó el ORM de Django, que abstrae gran parte de las operaciones de bajo nivel y permite trabajar con modelos en Python. La elección de SQL como tecnología complementaria al ORM de Django responde a criterios de compatibilidad, flexibilidad y control, lo que lo convirtió en un recurso indispensable frente a depender exclusivamente de la abstracción del ORM en el contexto específico del proyecto.

1.6.2 Lenguajes del lado del cliente (Frontend)

Los lenguajes del lado del cliente se ejecutan en el navegador del usuario y son responsables de la presentación visual, la interactividad y la experiencia de usuario.

HTML 5

El HyperText Markup Language (HTML) es un lenguaje de marcas hipertextuales, un lenguaje diseñado para estructurar textos para generar páginas web. La mayoría de las etiquetas del lenguaje HTML son semánticas. La interpretación de las etiquetas es realizada por el navegador web. Se utilizó el lenguaje HTML porque es extensible, además, se le pueden añadir características, etiquetas y funciones adicionales para el diseño de diseños, generando un producto vistoso, rápido y sencillo. [29]

JavaScript

JavaScript: Lenguaje interpretado que se ejecuta en el navegador, utilizado para dotar de interactividad y dinamismo a las páginas web. Su sintaxis se asemeja a Java y C, y se basa en prototipos en lugar de clases tradicionales. Gracias al acceso al DOM, puede modificar el contenido y comportamiento de la página en tiempo real.[30]

CSS

CSS (Cascading Style Sheets): Complementa al HTML definiendo el estilo visual de los elementos: colores, tipografías, márgenes, animaciones y disposición. Permite separar el contenido de su presentación, facilitando el diseño adaptable y profesional.[31]

1.6.3 Lenguaje de modelado unificado (UML)

UML (*Unified Modeling Language*, por sus siglas en inglés), es un estándar aprobado por la norma ISO (International Organization for Standardization), que, en la ingeniería del software, se utiliza como diagramado para representar el comportamiento y las funcionalidades de un programa. Es un lenguaje gráfico para visualizar, especificar, construir y documentar un sistema. UML ofrece un estándar para describir un plano del sistema (modelo), incluyendo aspectos conceptuales tales como procesos de negocio y funciones del sistema, aspectos concretos como expresiones de lenguajes de programación, esquemas de bases de datos y componentes reutilizables.[32] Los diagramas UML poseen las siguientes características:

- Muestran una idea general del software a desarrollar.
- Muestran posibles comportamientos que podría tener el programa al ejecutar tareas específicas

- Dan una perspectiva sobre todos los aspectos que quiere cubrir la aplicación.
- Permiten mejorar o corregir posibles inconformidades o puntos imprecisos en el levantamiento de los requisitos.
- Permiten visualizar y realizar los ajustes necesarios en la etapa de desarrollo.

Se utilizó el lenguaje de modelado UML pues permite que el modelo y el código estén actualizados, manteniendo la visión en el diseño de la estructura de un proyecto.

1.7 Conclusiones del capítulo

En este capítulo, se han analizado conceptos generales que han sido fundamentales para proporcionar soporte y mejorar la comprensión del desarrollo de este software de gestión de miembros en organizaciones y asociaciones. Se llevó a cabo un estudio sobre los sistemas de gestión implementados en diversas asociaciones y organizaciones, destacando la importancia de tener un sistema que maneje la información relacionada con los componentes más esenciales de estas, como son los miembros que las componen. Este estudio sirvió de punto de partida para la investigación. Además, se detallan las principales tecnologías y herramientas utilizadas, junto con la metodología de desarrollo de software empleada, explicando las razones para la selección de cada una de ellas. Estas herramientas y tecnologías se seleccionaron para asegurar que el proceso de gestión de asociados sea eficiente, eficaz y de alta calidad.

CAPÍTULO II DESCRIPCIÓN DE LA SOLUCIÓN PROPUESTA

2.1 Introducción

En este capítulo se hace referencia a las principales características de del sistema a desarrollar. Además, se definen los requerimientos del sistema mediante las historias de usuario (HU), así como aspectos relacionados con el diseño del software. En un proceso de desarrollo de software es importante usar métodos y técnicas que permitan la solución de problemas existentes en el ciclo de vida de este. La correcta captura de requisitos y el modelado del sistema permiten que se mitiguen las fallas que puedan aparecer durante su desarrollo, además de establecer un entendimiento común entre lo que se desea y lo que se elabora. Es por eso por lo que la producción de software debe cumplir tareas organizadas y con un orden lógico entre ellas.

Las organizaciones y/o asociaciones, debido a su creciente tamaño y complejidad, necesitan informatizar sus procesos para mejorar la eficiencia y eficacia en sus operaciones. Es imperativo que las estructuras de trabajo cuenten con sistemas informáticos funcionales, que faciliten las tareas de los empleados y de los directivos. Por esta razón, la implementación de un software de gestión es esencial para garantizar un aumento en la calidad y en la productividad del trabajo.

2.2 Descripción del proceso de gestión de los asociados.

La gestión de los asociados en Organizaciones y/o Asociaciones es un proceso crítico y de vital importancia para su funcionamiento. En el Comité Provincial de la UNEAC de Santiago de Cuba este proceso es realizado por la oficina de atención a filiales, este proceso se realiza de forma manual auxiliándose de programas como Microsoft Word y Excel para la realización del proceso de gestión.

Registro del asociado: El primer paso en la gestión de asociados implica registrar en el *Libro de Registro de Miembros* a aquellos asociados que han sido aprobados a través del proceso de Crecimiento supervisado por el Secretariado Nacional. Para optimizar el manejo de expedientes, se elabora una síntesis de los datos en una tarjeta en formato de papel, evitando el uso frecuente del expediente completo del asociado. Durante este proceso de registro, se recopilan datos relevantes tanto del asociado como de su membresía, incluyendo, entre otros:

Datos relevantes del asociado

- | | |
|---------------------------|-------------------------|
| - Nombre | - Dirección particular |
| - Segundo Nombre | - Móvil |
| - Primer Apellido | - Teléfono |
| - Segundo Apellido | - E-mail |
| - Fecha de nacimiento | - Nivel Escolaridad |
| - Sexo | - Ocupación |
| - No. Carnet de Identidad | - No. Carnet de miembro |

Datos relevantes a la membresía

- Fecha de Ingreso
- Fecha de baja (argumentado el motivo)
- Filial que pertenece
- Ejecutivo Comité Provincial (presidente, vicepresidente primero, vicepresidente)
- Ejecutivo de la Filial (presidente, vicepresidente, secretario)
- Miembro de Comisiones de trabajo permanente

Clasificación del asociado: Una vez recopilados los datos de los asociados, estos son clasificados según la filial a la que pertenecen. Para esta clasificación, se consideran únicamente los datos pertinentes, que se almacenan en una hoja electrónica utilizando el software Microsoft Excel.

- | | |
|--------------------|------------------------|
| - Nombre | - Carnet de identidad |
| - Segundo Nombre | - Dirección Particular |
| - Primer Apellido | - Móvil |
| - Segundo Apellido | - Teléfono |
| - Sexo | - Ocupación |

Registro de pagos de cuota de cotización: Se lleva a cabo un registro detallado en un documento de texto elaborado con Microsoft Word, donde se consignan los pagos realizados por los asociados. Este registro proporciona un seguimiento preciso de los pagos, detallando tanto la fecha de cada pago como los meses que cubre la cotización del asociado.

Creación de reportes: Con la información almacenada y clasificada de los asociados, se elaboran reportes en documentos de Word. Estos reportes permiten

un seguimiento detallado y una presentación organizada de los datos, facilitando la toma de decisiones y la gestión efectiva de los asociados tanto por la secretaría de filiales como por la administración de la organización. Algunos ejemplos de estos reportes incluyen:

- | | |
|-------------------------------------|----------------------------------|
| - Miembros totales por filiales | - Premios y condecoraciones |
| - Ejecutivo del Comité Provincial | - Morosos en pagos de cotización |
| - Ejecutivo de las filiales | - Actividades del mes |
| - Comisiones permanentes de trabajo | - Bajas del Comité Provincial |
| - Cumpleaños del mes | |

Calendario de eventos y actividades: El calendario constituye un cronograma detallado que especifica el nombre del evento, actividad o reunión, junto con el lugar, la fecha y la hora de realización.

- En el caso de actividades o eventos, se indica la filial responsable de su organización.
- Para los eventos, se consignan la fecha de inicio y la de finalización.
- En las reuniones, se precisa si corresponden al ejecutivo provincial, al ejecutivo de la filial o a alguna comisión de trabajo permanente.

Este calendario funciona como una herramienta organizativa e informativa esencial, que facilita la planificación y coordinación, además de agilizar el envío de invitaciones a los asociados involucrados. De esta manera, se garantiza que todos los participantes estén debidamente informados y puedan asistir puntualmente a las actividades programadas.

2.3 Propuesta del sistema

A partir del diagnóstico de las problemáticas actuales en la gestión administrativa de la Secretaría de Filiales, se propone el desarrollo de una aplicación web orientada a la administración integral de asociados. La solución tecnológica contempla la implementación de una base de datos centralizada para el registro y actualización de la información de miembros, la automatización de tareas rutinarias y la incorporación de una interfaz gráfica basada en principios de diseño UX/UI que favorezca la usabilidad y la accesibilidad.

Con ello se busca garantizar una mayor eficiencia operativa con una reducción ponderada de los tiempos de gestión superior al 95%, tal como se evidencia en la Tabla 2.1, así como asegurar la trazabilidad de los datos y la interoperabilidad con sistemas heredados mediante la exportación a formatos estándar como PDF. Esta propuesta responde a la necesidad de optimizar procesos críticos, reducir la carga administrativa y fortalecer la confiabilidad institucional en la gestión de información.

Tabla 2 1 Estimación de mejora de tiempos de gestión

Proceso	Tiempo Actual (minutos)	Tiempo Estimado con Sistema (minutos)	Reducción %
<i>Registro de asociado</i>	5	2	60%
<i>Actualización de datos del asociado</i>	12	0.50	95.83%
<i>Generación de reporte de cumpleaños del mes</i>	40	0.30	99.25%
Promedio ponderado	95.09%		

El cálculo del promedio ponderado de reducción se realizó utilizando los tiempos actuales como pesos. En lugar de promediar directamente los porcentajes de reducción, se multiplicó cada porcentaje por el tiempo actual correspondiente y luego se dividió entre la suma total de los tiempos actuales (57 minutos). De esta forma, los procesos de mayor duración tienen mayor influencia en el resultado global, lo que permite reflejar con precisión la eficiencia integral del sistema.

2.4 Roles y responsabilidades.

Durante el desarrollo del software se definieron roles y responsabilidades, que a continuación se describen.

Tabla 2 2 Roles y Responsabilidades

Roles	Nombre y Apellidos	Clasificación	Descripción
Directora de Investigación	Idalma Pérez Estévez	Especialista	Coordina y dirige la ejecución de los requisitos definidos durante la fase de investigación del proceso de gestión de asociados.
Cliente	Comité Provincial de la Uneac en Santiago de Cuba	Entidad	Establece los elementos a desarrollar y proporciona los datos necesarios para la elaboración de los requisitos funcionales y las historias técnicas del negocio.
Programador	Hirsel Paredes Vega	Estudiante	Responsable de tomar decisiones técnicas, construir el sistema y diseñar y ejecutar las pruebas.
Analista	Hirsel Paredes Vega	Estudiante	Reúne toda la información referente al negocio y especifica los requisitos del negocio en cada fase de desarrollo.
Diseñador	Hirsel Paredes Vega	Estudiante	Responsable de todo el diseño del software, asegurando que cumpla con los requisitos especificados por el cliente.
Arquitecto	Hirsel Paredes Vega	Estudiante	Define las partes integrantes del sistema.
Probador	Hirsel Paredes Vega	Estudiante	Ejecuta las pruebas y difunde los resultados de estas.

2.5 Usuarios del sistema.

Son todas aquellas personas que interactúan de alguna forma con el sistema o que desempeñan un rol específico en la operatividad de este. En la aplicación, los roles se representan de acuerdo con privilegios o permisos, es decir, diversas categorías de accesibilidad que determinan un nivel de operaciones dentro de las funcionalidades de esta.[33] En este caso, se definieron tres roles de usuario, que se muestran a continuación:

Tabla 2 3 Usuarios del sistema

Rol		Descripción
	Administrador	Acceso completo a todas las funcionalidades del sistema.
	Secretario	Responsabilidad de gestionar todas las funcionalidades relacionadas con los asociados.
	Consultor	Acceso limitado, restringido a la visualización de la información disponible en el sistema.

2.6 Funcionalidades del sistema

En Extreme Programming, las funcionalidades del sistema se definen a través de Historias de Usuario (HU), que capturan los requisitos desde la perspectiva del usuario final. Cada historia representa una unidad de valor entregable y se prioriza colaborativamente con el cliente.[34] A diferencia de los documentos tradicionales de requisitos, en XP estas historias evolucionan mediante conversación constante y se validan con pruebas de aceptación.

Requisitos funcionales del sistema:

- HU_1: Iniciar sesión
- HU_2: Gestionar Asociados
- HU_3: Gestionar Cotizaciones
- HU_4: Gestionar Calendario de Actividades
- HU_5: Gestionar Usuarios del Sistema
- HU_6: Consultar Asociados
- HU_7: Listar Miembros del Comité Provincial.
- HU_8: Listar bajas del Comité Provincial
- HU_9: Listar Ejecutivo del Comité Provincial.
- HU_10: Listar Ejecutivo de Filiales.
- HU_11: Listar Comisiones de Trabajo.
- HU_12: Listar Premios y Distinciones
- HU_13: Listar Actividades del Comité Provincial
- HU_14: Listar Cumpleaños por mes
- HU_15: Listar Morosos de pagos de Cotizaciones
- HU_16: Exportar reportes a PDF
- HU_17: Visualizar Distribución de Asociados por Filial (gráfico)
- HU_18: Visualizar Distribución por Género y Filial (gráfico)
- HU_19: Visualizar Distribución por Grupos de Edad (gráfico)
- HU_20: Visualizar Perfil Demográfico (gráfico)
- HU_21: Visualizar Nivel de Formación Académica (gráfico)
- HU_22: Visualizar Bajas por Filial (gráfico)
- HU_23: Visualizar Actividades por Filial (gráfico)

2.7 Historias de Usuario

Las historias de usuario (HU) también conocidas como “solicitudes del usuario” en la metodología de desarrollo XP son un mecanismo de representación de un requerimiento de software desde el punto de vista del cliente, es decir, son un conjunto de funcionalidades que el cliente desea implementar en el futuro sistema. Estas HU representan una forma rápida de administrar los requerimientos de los usuarios sin tener que elaborar documentos formales y sin requerir mucho tiempo para administrarlos.[34]

A continuación, se presentan algunas de las historias de usuario implementadas.

Tabla 2 4 Historia de Usuario: “Iniciar Sesión”

Historia de Usuario	
Numero: 1	Nombre Historia de Usuario: Iniciar sesión
Modificación o Extensión de Historia de Usuario y Número: Ninguna	
Usuario: Todos los usuarios	Iteración asignada: 1
Prioridad en el negocio: Alta	Puntos estimados: 4 días
Riesgo en desarrollo: Medio	Puntos reales: 4 días
Programador responsable: Hirsal Paredes Vega	
Descripción: Como usuario del sistema (Administrador, Secretario o Consultor), quiero iniciar sesión mediante un formulario con usuario y contraseña, para acceder a las funcionalidades correspondientes a mi rol.	
Observaciones: Si las credenciales son incorrectas, el sistema mostrará un mensaje de error y no permitirá el acceso.	

Tabla 2 5 Historia de Usuario: “Gestionar Asociados”

Historia de Usuario	
Numero: 2	Nombre Historia de Usuario: Gestionar Asociados
Modificación o Extensión de Historia de Usuario y Número: Ninguna	
Usuario: Administrador, Secretario	Iteración asignada: 1
Prioridad en el negocio: Alta	Puntos estimados: 5 días
Riesgo en desarrollo: Alto	Puntos reales: 5 días
Programador responsable: Hirsal Paredes Vega	

Descripción: Como Administrador o Secretario, quiero registrar, editar, consultar y eliminar la información de los asociados, para mantener actualizada la base de datos centralizada y garantizar la trazabilidad de los miembros de la organización.

Observaciones: Solo los roles de Administrador y Secretario podrán gestionar asociados.

Tabla 2 6 Historia de Usuario: “Gestionar Cotizaciones”

Historia de Usuario	
Numero: 3	Nombre Historia de Usuario: Gestionar Cotizaciones
Modificación o Extensión de Historia de Usuario y Número: Ninguna	
Usuario: Administrador, Secretario	Iteración asignada: 2
Prioridad en el negocio: Alta	Puntos estimados: 5 días
Riesgo en desarrollo: Alto	Puntos reales: 5 días
Programador responsable: Hirsell Paredes Vega	
Descripción: Como Administrador o Secretario, quiero registrar, editar y eliminar cotizaciones de los asociados, para mantener actualizada la información de pagos y garantizar la trazabilidad financiera de la organización.	
Observaciones: Esta opción solo estará disponible una vez el usuario este autenticado y tenga el rol de Administrador o Secretario	

Tabla 2 7 Historia de Usuario: “Gestionar Usuarios del Sistema”

Historia de Usuario	
Numero: 5	Nombre Historia de Usuario: Gestionar Usuarios del Sistema
Modificación o Extensión de Historia de Usuario y Número: Ninguna	
Usuario: Administrador	Iteración asignada: 2
Prioridad en el negocio: Alta	Puntos estimados: 3 días
Riesgo en desarrollo: Media	Puntos reales: 3 días
Programador responsable: Hirsell Paredes Vega	

Descripción: Como Administrador, quiero crear, editar y eliminar cuentas de usuario, asignando roles específicos (Administrador, Secretario, Consultor), para asegurar el control de accesos y la correcta administración del sistema.

Observaciones: Solo el rol de Administrador tendrá acceso a la gestión de usuarios.

2.8 Tareas de Ingeniería

Las tareas de ingeniería representan los pasos concretos y específicos necesarios para implementar una historia de usuario (HU). Estas tareas permiten descomponer una HU en acciones más pequeñas y manejables, facilitando su ejecución, seguimiento y control. Una historia de usuario se divide en varias tareas de ingeniería, las cuales deben ser lo suficientemente pequeñas, específicas y acotadas en tiempo para poder ser completadas en un período corto, idealmente en unas pocas horas. Las tareas mostradas a continuación representan las principales necesarias para la implementación del sistema. [34]

Tabla 2 8 Tarea de Ingeniería: “Diseño del formulario de inicio de sesión”

Tarea de Ingeniería	
Numero 1	Historia de usuario: HU_1 Iniciar Sesión
Nombre de Tarea: Diseño del formulario de inicio de sesión	
Tipo de Tarea: (Desarrollo/Corrección/Mejora): Diseño	Puntos Estimados: 2 día
Fecha de Inicio: 05/01/25	Fecha de Finalización: 07/01/25
Riesgo en desarrollo (Alta/Media/ Baja): Media	Puntos Reales: 0.5 día
Programador responsable: Hirsal Paredes Vega	
Descripción: Crear el formulario con campos de usuario y contraseña, aplicando estilos CSS para accesibilidad y usabilidad.	

Tabla 2 9Tarea de Ingeniería: “Implementación de la lógica de autenticación”

Tarea de Ingeniería	
Numero 2	Historia de usuario: HU_1 Iniciar Sesión
Nombre de Tarea: Implementación de la lógica de autenticación	

Tipo de Tarea: <i>(Desarrollo/Corrección/Mejora):</i> Desarrollo	Puntos Estimados: 3 día
Fecha de Inicio: 08/01/25	Fecha de Finalización: 11/01/25
Riesgo en desarrollo <i>(Alta/Media/ Baja):</i> Alta	Puntos Reales: 1.5 días
Programador responsable: Hirsell Paredes Vega	
Descripción: Programar la vista en Django para validar credenciales contra la base de datos y gestionar sesiones seguras.	

Tabla 2 10Tarea de Ingeniería: “Configuración de roles y permisos”

Tarea de Ingeniería	
Numero 3	Historia de usuario: HU_1 Iniciar Sesión
Nombre de Tarea: “Configuración de roles y permisos”	
Tipo de Tarea: <i>(Desarrollo/Corrección/Mejora):</i> Desarrollo	Puntos Estimados: 2 día
Fecha de Inicio: 12/01/25	Fecha de Finalización: 14/01/25
Riesgo en desarrollo <i>(Alta/Media/ Baja):</i> Media	Puntos Reales: 0.5 día
Programador responsable: Hirsell Paredes Vega	
Descripción: Definir roles (Administrador, Secretario, Consultor) y asignar permisos diferenciados según el rol autenticado.	

Tabla 2 11 Tarea de Ingeniería: “Validaciones y mensajes de error”

Tarea de Ingeniería	
Numero 4	Historia de usuario: HU_1 Iniciar Sesión
Nombre de Tarea: “Validaciones y mensajes de error”	
Tipo de Tarea: <i>(Desarrollo/Corrección/Mejora):</i> Desarrollo	Puntos Estimados: 1 día
Fecha de Inicio: 15/01/25	Fecha de Finalización: 16/01/25
Riesgo en desarrollo <i>(Alta/Media/ Baja):</i> Baja	Puntos Reales: 0.5 día
Programador responsable: Hirsell Paredes Vega	
Descripción: Implementar validaciones de campos obligatorios y mostrar mensajes de error cuando las credenciales sean incorrectas.	

Tabla 2 12 Tarea de Ingeniería: “Seguridad en almacenamiento de contraseñas”

Tarea de Ingeniería	
Numero 5	Historia de usuario: HU_1 Iniciar Sesión
Nombre de Tarea: “Seguridad en almacenamiento de contraseñas”	
Tipo de Tarea: <i>(Desarrollo/Corrección/Mejora):</i> Desarrollo	Puntos Estimados: 2 día
Fecha de Inicio: 17/01/25	Fecha de Finalización: 19/01/25
Riesgo en desarrollo <i>(Alta/Media/ Baja):</i> Alta	Puntos Reales: 1 día
Programador responsable: Hirsal Paredes Vega	
Descripción: Configurar cifrado de contraseñas en la base de datos y limitar intentos de acceso para prevenir ataques de fuerza bruta.	

Tabla 2 13 Tarea de Ingeniería: “Implementación del modelo Django para asociados”

Tarea de Ingeniería	
Numero 6	Historia de usuario: HU_2 Gestionar Asociados
Nombre de Tarea: Implementación del modelo Django para asociados	
Tipo de Tarea: <i>(Desarrollo/Corrección/Mejora):</i> Desarrollo	Puntos Estimados: 2 día
Fecha de Inicio: 20/01/25	Fecha de Finalización: 21/01/25
Riesgo en desarrollo <i>(Alta/Media/ Baja):</i> Media	Puntos Reales: 1.5 día
Programador responsable: Hirsal Paredes Vega	
Descripción: Programar el modelo Django que represente a los asociados, definiendo atributos, relaciones y restricciones de integridad.	

Tabla 2 14 Tarea de Ingeniería: “Desarrollo de vistas CRUD para asociados”

Tarea de Ingeniería	
Numero 7	Historia de usuario: HU_2 Gestionar Asociados
Nombre de Tarea: Desarrollo de vistas CRUD para asociados	
Tipo de Tarea: <i>(Desarrollo/Corrección/Mejora):</i> Desarrollo	Puntos Estimados: 2 día
Fecha de Inicio: 22/01/25	Fecha de Finalización: 23/01/25

Riesgo en desarrollo (<i>Alta/Media/ Baja</i>): Alta	Puntos Reales: 1.5 día
Programador responsable: Hirscl Paredes Vega	
Descripción: Implementar las vistas en Django para crear, consultar, editar y eliminar asociados, integrando el modelo con la base de datos.	

Tabla 2 15 Tarea de Ingeniería: “Diseño de formularios y validaciones de datos”

Tarea de Ingeniería	
Numero 8	Historia de usuario: HU_2 Gestionar Asociados
Nombre de Tarea: Diseño de formularios y validaciones de datos	
Tipo de Tarea: (<i>Desarrollo/Corrección/Mejora</i>): Desarrollo/Diseño	Puntos Estimados: 1 día
Fecha de Inicio: 24/01/25	Fecha de Finalización: 24/01/25
Riesgo en desarrollo (<i>Alta/Media/ Baja</i>): Baja	Puntos Reales: 0.5 día
Programador responsable: Hirscl Paredes Vega	
Descripción: Crear formularios HTML5/CSS3 para la gestión de asociados, implementando validaciones de campos obligatorios y reglas de negocio.	

Tabla 2 16 Tarea de Ingeniería: “Configuración de permisos por rol”

Tarea de Ingeniería	
Numero 9	Historia de usuario: HU_2 Gestionar Asociados
Nombre de Tarea: Configuración de permisos por rol	
Tipo de Tarea: (<i>Desarrollo/Corrección/Mejora</i>): Desarrollo	Puntos Estimados: 1 día
Fecha de Inicio: 25/01/25	Fecha de Finalización: 25/01/25
Riesgo en desarrollo (<i>Alta/Media/ Baja</i>): Media	Puntos Reales: 0.5 día
Programador responsable: Hirscl Paredes Vega	
Descripción: Configurar permisos en Django para que solo los roles de Administrador y Secretario puedan gestionar asociados.	

Tabla 2 17 Tarea de Ingeniería: “Registro de auditoría para cambios en asociados”

Tarea de Ingeniería			
Numero 10		Historia de usuario: HU_2 Gestionar Asociados	
Nombre de Tarea: Registro de auditoría para cambios en asociados			
Tipo de Tarea: (Desarrollo/Corrección/Mejora): Desarrollo		Puntos Estimados: 1 día	
Fecha de Inicio: 26/01/25		Fecha de Finalización: 26/01/25	
Riesgo en desarrollo (Alta/Media/ Baja): Media		Puntos Reales: 1 día	
Programador responsable: Hirsel Paredes Vega			
Descripción: Implementar un sistema de auditoría que registre las operaciones realizadas sobre los asociados (alta, edición, baja).			

Tabla 2 18 Tarea de Ingeniería: “Implementación del modelo Django para cotizaciones”

Tarea de Ingeniería		
Numero 11	Historia de usuario: HU_3 Gestionar Cotizaciones	
Nombre de Tarea: Implementación del modelo Django para cotizaciones		
Tipo de Tarea: (Desarrollo/Corrección/Mejora): Desarrollo		Puntos Estimados: 2 día
Fecha de Inicio: 27/01/25		Fecha de Finalización: 28/01/25
Riesgo en desarrollo (Alta/Media/ Baja): Media		Puntos Reales: 1 día
Programador responsable: Hirsel Paredes Vega		
Descripción: Programar el modelo Django que represente las cotizaciones, definiendo atributos (monto, fecha, estado) y relaciones con asociados.		

Tabla 2 19 Tarea de Ingeniería: “Desarrollo de vistas CRUD para cotizaciones”

Tarea de Ingeniería		
Numero 13	Historia de usuario: HU_3 Gestionar Cotizaciones	
Nombre de Tarea: Desarrollo de vistas CRUD para cotizaciones		
Tipo de Tarea: (Desarrollo/Corrección/Mejora): Desarrollo		Puntos Estimados: 2 días
Fecha de Inicio: 29/01/25		Fecha de Finalización: 30/01/25

Riesgo en desarrollo (<i>Alta/Media/ Baja</i>): Alta	Puntos Reales: 1.5 día
Programador responsable: Hirscl Paredes Vega	
Descripción: Implementar las vistas en Django para registrar, consultar, editar y eliminar cotizaciones, integrando el modelo con la base de datos.	

Tabla 2 20 Tarea de Ingeniería: “Diseño de formularios y validaciones de datos”

Tarea de Ingeniería	
Numero 14	Historia de usuario: HU_3 Gestionar Cotizaciones
Nombre de Tarea: Diseño de formularios y validaciones de datos	
Tipo de Tarea: (<i>Desarrollo/Corrección/Mejora</i>): Desarrollo/Diseño	Puntos Estimados: 1 día
Fecha de Inicio: 31/01/25	Fecha de Finalización: 30/01/25
Riesgo en desarrollo (<i>Alta/Media/ Baja</i>): Baja	Puntos Reales: 0.5 día
Programador responsable: Hirscl Paredes Vega	
Descripción: Crear formularios HTML5/CSS3 para la gestión de cotizaciones, implementando validaciones de campos obligatorios y reglas de negocio.	

Tabla 2 21 Tarea de Ingeniería: “Configuración de permisos por rol”

Tarea de Ingeniería	
Numero 15	Historia de usuario: HU_3 Gestionar Cotizaciones
Nombre de Tarea: Configuración de permisos por rol	
Tipo de Tarea: (<i>Desarrollo/Corrección/Mejora</i>): Desarrollo	Puntos Estimados: 1 día
Fecha de Inicio: 01/02/25	Fecha de Finalización: 01/02/25
Riesgo en desarrollo (<i>Alta/Media/ Baja</i>): Media	Puntos Reales: 0.5 día
Programador responsable: Hirscl Paredes Vega	
Descripción: Configurar permisos en Django para que solo los roles de Administrador y Secretario puedan gestionar cotizaciones.	

Tabla 2 22 Tarea de Ingeniería: “Registro de auditoría para operaciones de cotización”

Tarea de Ingeniería			
Numero 16		Historia de usuario: HU_3 Gestionar Cotizaciones	
Nombre de Tarea: Registro de auditoría para operaciones de cotización			
Tipo de Tarea: (Desarrollo/Corrección/Mejora): Desarrollo		Puntos Estimados: 1 día	
Fecha de Inicio: 02/02/25		Fecha de Finalización: 02/02/25	
Riesgo en desarrollo (Alta/Media/ Baja): Media		Puntos Reales: 1.5 día	
Programador responsable: Hirsell Paredes Vega			
Descripción: Implementar un sistema de auditoría que registre las operaciones realizadas sobre las cotizaciones (alta, edición, baja).			

Tabla 2 23 Tarea de Ingeniería: “Implementación del modelo Django para usuarios y roles”

Tarea de Ingeniería			
Numero 17	Historia de usuario: HU_5 Gestionar Usuarios del Sistema		
Nombre de Tarea: Implementación del modelo Django para usuarios y roles			
Tipo de Tarea: (Desarrollo/Corrección/Mejora): Desarrollo		Puntos Estimados: 2 día	
Fecha de Inicio: 03/02/25		Fecha de Finalización: 04/02/25	
Riesgo en desarrollo (Alta/Media/ Baja): Media		Puntos Reales: 0.5 día	
Programador responsable: Hirsel Paredes Vega			
Descripción: Programar el modelo Django que represente a los usuarios, definiendo atributos básicos y roles asociados (Administrador, Secretario, Consultor).			

Tabla 2 24 Tarea de Ingeniería: “Desarrollo de vistas CRUD para usuarios”

Tarea de Ingeniería	
Numero 18	Historia de usuario: HU_5 Gestionar Usuarios del Sistema
Nombre de Tarea: Desarrollo de vistas CRUD para usuarios	

Tipo de Tarea: <i>(Desarrollo/Corrección/Mejora):</i> Desarrollo	Puntos Estimados: 2 día
Fecha de Inicio: 05/02/25	Fecha de Finalización: 06/02/25
Riesgo en desarrollo <i>(Alta/Media/ Baja):</i> Alta	Puntos Reales: 0.5 día
Programador responsable: Hirsal Paredes Vega	
Descripción: Implementar las vistas en Django para crear, consultar, editar y eliminar usuarios, integrando el modelo con la base de datos.	

Tabla 2 25 Tarea de Ingeniería: “Asignación de roles y permisos”

Tarea de Ingeniería	
Numero 19	Historia de usuario: HU_5 Gestionar Usuarios del Sistema
Nombre de Tarea: “Asignación de roles y permisos”	
Tipo de Tarea: <i>(Desarrollo/Corrección/Mejora):</i> Desarrollo	Puntos Estimados: 1 día
Fecha de Inicio: 07/02/25	Fecha de Finalización: 07/02/25
Riesgo en desarrollo <i>(Alta/Media/ Baja):</i> Media	Puntos Reales: 0.5 día
Programador responsable: Hirsal Paredes Vega	
Descripción: Configurar permisos en Django para que cada rol tenga acceso únicamente a las funcionalidades correspondientes.	

Tabla 2 26 Tarea de Ingeniería: “Validaciones de seguridad”

Tarea de Ingeniería	
Numero 20	Historia de usuario: HU_5 Gestionar Usuarios del Sistema
Nombre de Tarea: Validaciones de seguridad	
Tipo de Tarea: <i>(Desarrollo/Corrección/Mejora):</i> Desarrollo	Puntos Estimados: 1 día
Fecha de Inicio: 08/02/25	Fecha de Finalización: 08/02/25
Riesgo en desarrollo <i>(Alta/Media/ Baja):</i> Alta	Puntos Reales: 1 día
Programador responsable: Hirsal Paredes Vega	
Descripción: Implementar validaciones de seguridad en contraseñas, bloqueo de cuentas y restablecimiento seguro de credenciales.	

Tabla 2 27 Tarea de Ingeniería: “Registro de auditoría para cambios en usuarios”

Tarea de Ingeniería		
Numero 21	Historia de usuario: HU_5 Gestionar Usuarios del Sistema	
Nombre de Tarea: Registro de auditoría para cambios en usuarios		
Tipo de Tarea: (Desarrollo/Corrección/Mejora): Desarrollo		Puntos Estimados: 1 día
Fecha de Inicio: 09/02/25		Fecha de Finalización: 09/02/25
Riesgo en desarrollo (Alta/Media/ Baja): Media		Puntos Reales: 0.5 día
Programador responsable: Hirsel Paredes Vega		
Descripción: Implementar un sistema de auditoría que registre las operaciones realizadas sobre los usuarios		

2.9 Historias Técnicas

Las funcionalidades del sistema se complementan mediante historias técnicas orientadas al diseño e implementación, las cuales definen las propiedades esenciales que debe poseer el producto para garantizar su atractivo, usabilidad, rendimiento y confiabilidad.[34] Estas características son determinantes para el éxito del sistema y se agrupan en los siguientes requisitos:

A continuación, se muestran algunos de estos requisitos, divididos en categorías:

2.9.1 Requisitos de apariencia o interfaz externa.

La aplicación debe ofrecer una interfaz intuitiva, visualmente agradable y estructurada, que proporcione señales claras de navegación y facilite el acceso al contenido deseado por parte del usuario.

2.9.2 Requisitos de usabilidad

El sistema debe ser accesible para usuarios sin conocimientos técnicos avanzados de informática, por lo que se requiere una experiencia de uso flexible, clara y funcional.

2.9.3 Requisitos de confidencialidad

Se establecerán diferentes niveles de acceso (roles) para los diferentes usuarios que interactúen con el sistema, lo que garantiza que la información manejada esté

protegida de acceso no autorizado y divulgación. Se implementan auditorías de acceso, trazabilidad de modificaciones y reportes de seguridad.

2.9.4 Requisitos de fiabilidad

La información gestionada por el sistema debe ser íntegra y confiable, con mecanismos de recuperación rápida ante fallos que aseguren la continuidad operativa.

2.9.5 Requisitos de Disponibilidad

El sistema debe permitir acceso oportuno a la información requerida en cualquier momento.

2.9.6 Requisitos de Software

El entorno del usuario final debe contar con sistemas operativos Windows 7 o superior, Linux Ubuntu 20.04+ o macOS 10.15+, y navegadores compatibles con HTML5, CSS3 y JavaScript ES6 como Chrome 90+, Firefox 88+, Safari 14+ o Edge 90+. Se requiere conectividad LAN estable para acceso remoto y correcta interacción con la aplicación.

2.9.7 Requisitos de Hardware

Cliente/Estación de trabajo: Procesador de al menos dos núcleos, 2 GB de RAM, 500 MB libres para caché, pantalla mínima de 1024×768 px y conexión Ethernet ≥10 Mbps.

Servidor local: Procesador de 2 núcleos, 8 GB de RAM, 50 GB SSD, red Gigabit Ethernet.

2.9.8 Requisitos de Seguridad

El sistema implementa un esquema de seguridad multicapa basado en las mejores prácticas de Django 5.2. Para el hashing de contraseñas, utiliza PBKDF2 con SHA256, algoritmo recomendado por NIST y configurado por defecto en Django, con 600,000 iteraciones para resistencia contra ataques de fuerza bruta. [35]

La protección contra CSRF (Cross-Site Request Forgery) está garantizada mediante el middleware `django.middleware.csrf.CsrfViewMiddleware`, que genera tokens únicos por sesión validados automáticamente en todas las operaciones de modificación. [35]

2.9.9 Requisitos de Rendimiento

El sistema debe ejecutar múltiples operaciones por segundo para dar una respuesta rápida al usuario y evitar demoras innecesarias. Los tiempos de respuesta serán entre 2 y 3 segundos para operaciones de recuperación, evitando demoras y asegurando fluidez en la interacción.

2.10 Diagrama de Clases

El diagrama de clases representa la estructura estática del sistema, modelando las entidades principales, sus atributos, métodos y relaciones. Este diseño permite visualizar la lógica orientada a objetos, facilitando la identificación de dependencias y asociaciones entre componentes clave como usuarios, asociados, módulos funcionales y registros de actividad. Su elaboración responde a principios de cohesión, reutilización y encapsulamiento, fundamentales para la mantenibilidad del sistema.[36] Los diagramas de clases son el pilar fundamental del modelado con UML, siendo ampliamente utilizados tanto para análisis como para diseño de sistemas y software en general.

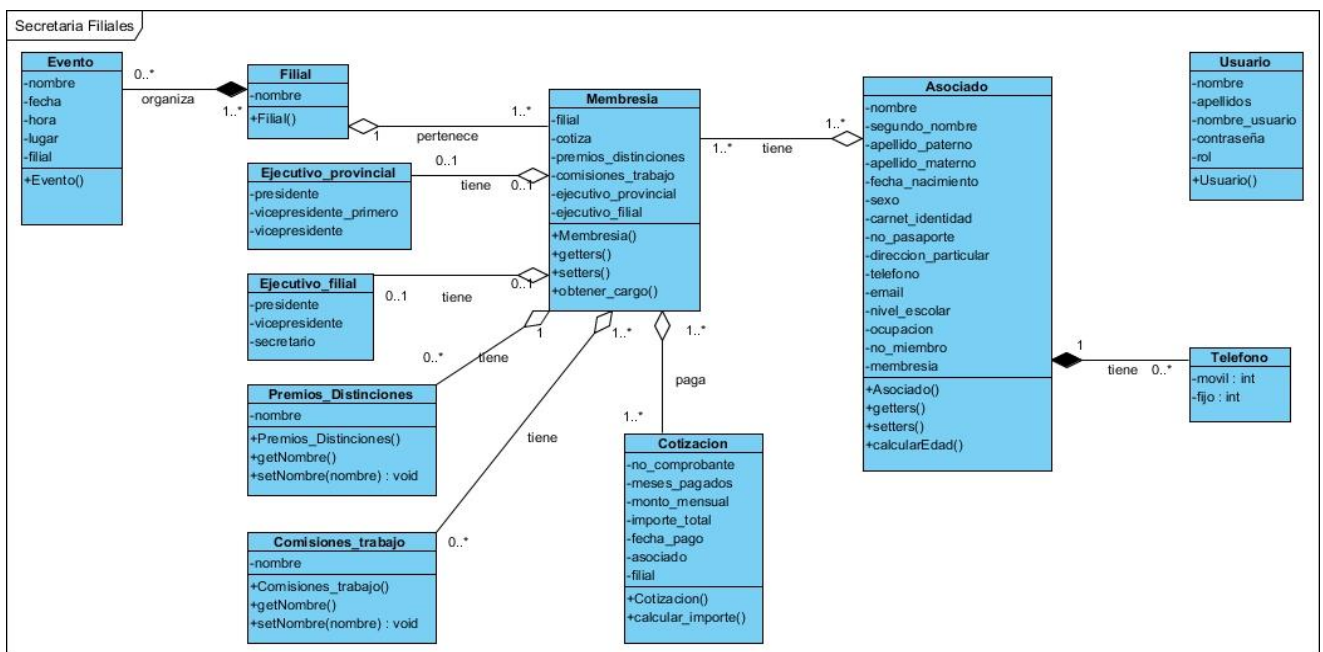


Figura 2. 1 Diagrama de Clases del diseño

En la figura 2.1 están representadas las clases que se identificaron durante el proceso de análisis y diseño del sistema, con sus correspondientes atributos y las relaciones que existen entre ellas. Como se observa, fue necesario implementar once (11) clases, entre las cuales se establecen distintos tipos de relaciones de

composición y agregación, así como multiplicidades que reflejan las reglas de negocio y la estructura organizativa del sistema.

El diseño orientado a objetos no solo se fundamenta en la correcta identificación de clases y relaciones, sino también en la asignación coherente de responsabilidades. Para ello, los patrones GRASP (Patrones Generales de Software para Asignar Responsabilidades) ofrecen lineamientos que permiten distribuir funciones de manera lógica, favoreciendo la cohesión interna de las clases, reduciendo el acoplamiento y asegurando la mantenibilidad del sistema. En el modelo institucional presentado, los patrones GRASP se aplican de forma natural, reforzando la claridad y robustez del diseño:

- Patrón Experto (Expert): asigna la responsabilidad de realizar una operación a la clase que posee la información necesaria. Ejemplo: la clase Cotizacion calcula su propio importe.
- Patrón Creador (Creator): se aplica cuando una clase necesita crear instancias de otra. Ejemplo: Asociado crea Telefono; Membresia crea Cotizacion.
- Patrón Alta Cohesión (High Cohesion): establece que una clase debe tener una única responsabilidad y métodos relacionados entre sí. Ejemplo: Telefono y Cotizacion.
- Patrón Bajo Acoplamiento (Low Coupling): busca reducir dependencias entre clases, de modo que un cambio en una no afecte a otra. Ejemplo: Telefono y Cotizacion.

La aplicación de estos patrones garantiza que el diseño sea más claro, escalable y fácil de mantener, alineando la estructura del sistema con buenas prácticas reconocidas en la ingeniería de software.

2.11 Diagrama de la Base de Datos

El diagrama de la base de datos representa la organización lógica de los datos del sistema institucional, incluyendo las tablas, sus campos, claves primarias y foráneas, así como las relaciones entre entidades. Se ha implementado una estructura relacional normalizada que garantiza la integridad referencial, la eficiencia en las consultas y la escalabilidad del modelo.[37] Las entidades modeladas abarcan componentes clave como:

- Usuarios (users_user)
- Asociados (asociados_asociado)
- Membresías (asociados_membresia)

- Cotizaciones (cotizaciones_cotizacion)
- Eventos (calendario_evento)
- Filiales y estructuras ejecutivas (asociados_filial, asociados_ejecutivofilial, asociados_ejecutivoprovincial)
- Comisiones y distinciones (asociados_membresia_distinciones, asociados_membresia_comisiones)

Estas entidades están alineadas con los requisitos funcionales, organizativos y de confidencialidad definidos previamente, permitiendo una trazabilidad clara de las operaciones y una gestión modular de los datos.

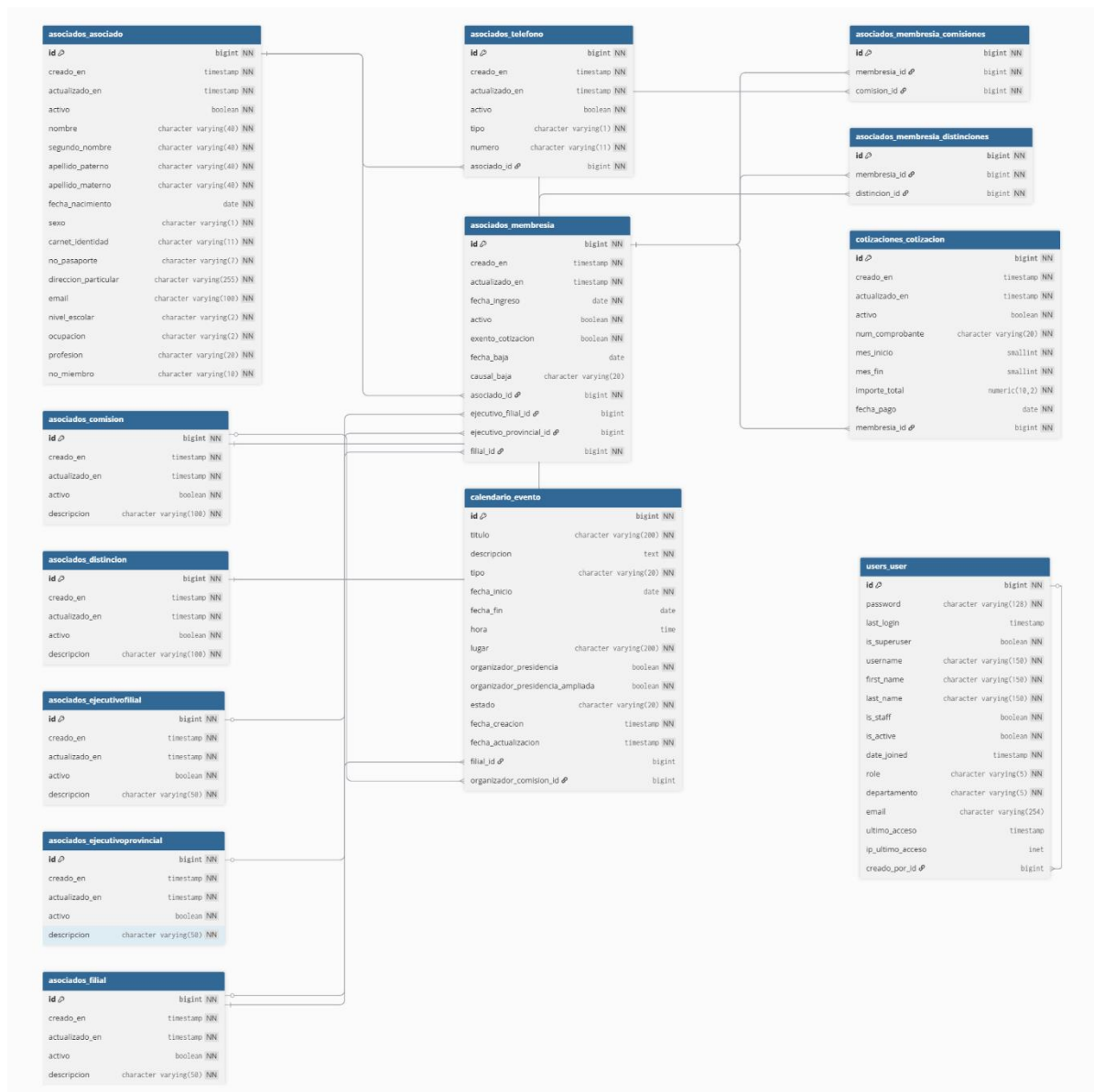


Figura 2. 2 Figura Diagrama de la Base de Datos

2.12 Normalización

La normalización es un proceso fundamental en el diseño de bases de datos relacionales, orientado a eliminar redundancias, evitar anomalías de actualización y garantizar la consistencia lógica de los datos. Consiste en la descomposición progresiva de las relaciones (tablas) en estructuras más simples, lo que puede aumentar la cantidad de tablas, pero mejora la claridad y eficiencia del modelo.[37] El diseño actual ha sido normalizado hasta la Tercera Forma Normal (3FN), cumpliendo con los siguientes criterios:

Primera Forma Normal (1FN): Todos los atributos contienen valores atómicos, es decir, indivisibles. No se permiten listas ni conjuntos dentro de un mismo campo.

Segunda Forma Normal (2FN): Todos los atributos no clave dependen completamente de la clave primaria de la tabla. No existen dependencias parciales en relaciones con claves compuestas.

Tercera Forma Normal (3FN): Los atributos no clave son independientes entre sí, es decir, no dependen funcionalmente de otros atributos no clave. Esto evita redundancias y facilita la actualización de datos.

Este nivel de normalización permite que el sistema sea más seguro, mantenible y extensible, facilitando la incorporación de nuevos módulos sin comprometer la integridad del modelo existente.

2.13 Tarjetas CRC

Las tarjetas CRC (Clase–Responsabilidad–Colaborador) se utilizan para representar de forma sintética el comportamiento de las clases dentro del sistema. Cada tarjeta identifica una clase, sus responsabilidades funcionales y las clases colaboradoras con las que interactúa.[38] Esta técnica facilita la validación temprana del diseño orientado a objetos, promoviendo la claridad en la asignación de funciones y la detección de acoplamientos innecesarios. A continuación, se presentan las tarjetas CRC correspondientes a las principales clases del sistema institucional:

Tabla 2 28 Tarjeta CRC

Clase: Asociado	
Responsabilidad	Colaboración
• Almacenar datos personales del miembro. • Gestionar sus teléfonos. • Asociarse a una o más membresías.	Telefono Membresia

Tabla 2 29 Tarjeta CRC

Clase: Membresia	
Responsabilidad	Colaboración
• Registrar inscripción del asociado. • Gestionar cotizaciones, comisiones, distinciones. • Determinar cargos ejecutivos ocupados.	Asociado Cotizacion EjecutivoFilial EjecutivoProvincial Comisiones Distinciones

Tabla 2 30 Tarjeta CRC

Clase: Cotizacion	
Responsabilidad	Colaboración
• Registrar pagos • Calcular importe total	Membresia

Tabla 2 31 Tarjeta CRC

Clase: Evento	
Responsabilidad	Colaboración
• Registrar actividades institucionales • Definir lugar, fecha y tipo	Filial Comisiones

De esta manera, las tarjetas CRC permiten visualizar de forma clara qué hace cada clase y con quién colabora, asegurando un diseño orientado a objetos coherente y mantenible.

2.14 Diagrama de Despliegue

El diagrama de despliegue representa la arquitectura física del sistema propuesto, en el cual la aplicación web desarrollada con el servidor WSGI Waitress y la base de datos residen en un único servidor físico.[38] Este enfoque responde a una estrategia de implementación monolítica, adecuada para entornos institucionales con recursos limitados y necesidades de mantenimiento centralizado.

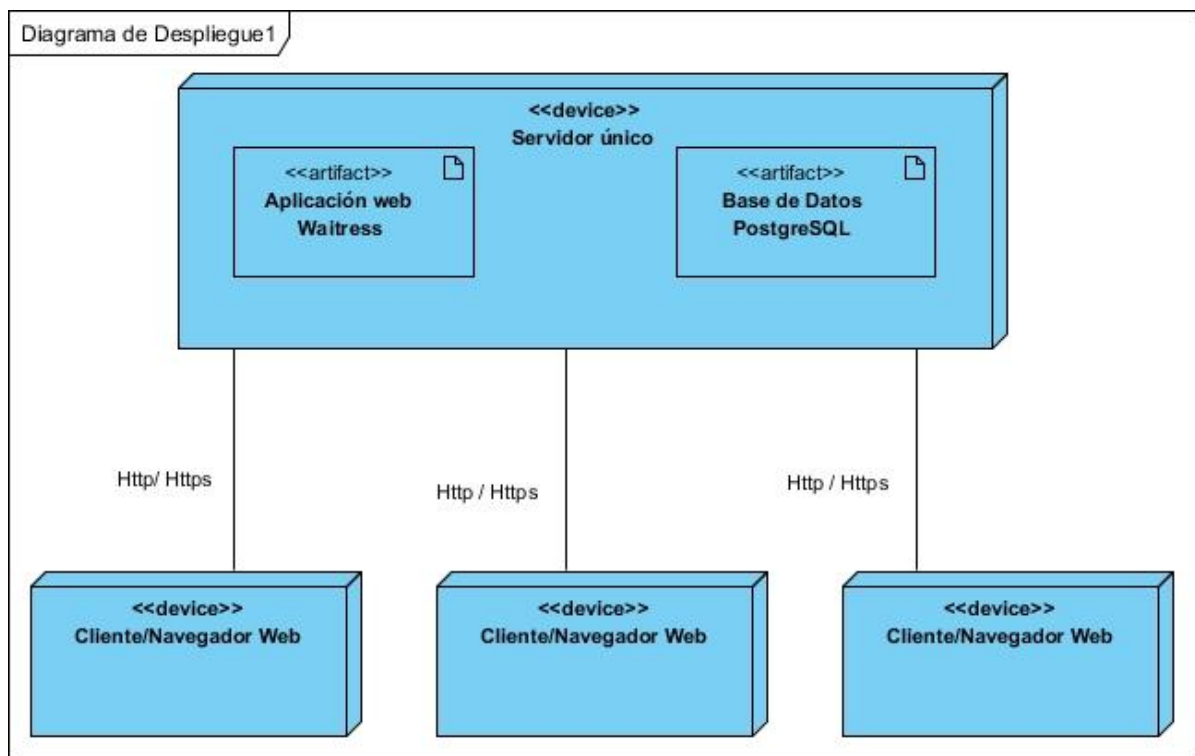


Figura 2. 3 Diagrama de despliegue

En el nodo identificado como `<<device>>` Servidor único, se despliegan dos artefactos principales:

`<<artifact>>` Aplicación web (Waitress): Encargada de gestionar las solicitudes HTTP/HTTPS provenientes de los clientes, ejecutar la lógica de negocio y coordinar el acceso a los datos.

`<<artifact>>` Base de Datos (PostgreSQL): Repositorio local que almacena la información estructurada del sistema, accesible directamente por la aplicación sin intermediación de red.

Los clientes, representados como nodos `<<device>>` Cliente/Navegador Web, interactúan exclusivamente a través del protocolo HTTP/HTTPS, accediendo a la interfaz web desde navegadores estándar. Esta configuración garantiza una comunicación segura y directa entre los usuarios y el servidor, sin necesidad de componentes intermedios ni balanceadores de carga.

La elección de Waitress como servidor de aplicaciones responde a su eficiencia en entornos WSGI, su facilidad de integración con frameworks Python y su capacidad para operar de forma estable en servidores modestos. Al mantener la base de datos

en el mismo nodo, se reduce la latencia de acceso y se simplifica la configuración inicial, aunque se reconoce que esta arquitectura podría evolucionar hacia una separación de capas en futuras fases de escalamiento.

2.15 Patrón de Arquitectura

La aplicación adopta el patrón de arquitectura MVT (Modelo–Vista–Template), propio del framework Django, que separa la lógica de negocio, la presentación y la estructura de las interfaces. Esta organización promueve el desarrollo modular, facilita la escalabilidad y mejora la mantenibilidad del sistema, aspectos especialmente pertinentes en el contexto institucional, donde la claridad en la separación de responsabilidades asegura la trazabilidad y la sostenibilidad del proyecto en el tiempo.[36] La adopción del MVT se integra de manera natural con la metodología Extreme Programming (XP), pues favorece la entrega incremental de funcionalidades y la refactorización continua, garantizando que la arquitectura se mantenga coherente y adaptable. Asimismo, la organización MVT se refleja en el despliegue monolítico del sistema, donde la aplicación gestionada por Waitress atiende las vistas y plantillas, mientras la base de datos respalda el modelo, consolidando la coherencia entre diseño lógico y físico. Este patrón se complementa con prácticas de diseño seguro, control de acceso por roles y gestión eficiente de recursos estáticos y dinámicos, asegurando que la solución responda tanto a las necesidades técnicas como a los requerimientos institucionales de confiabilidad y legitimidad.

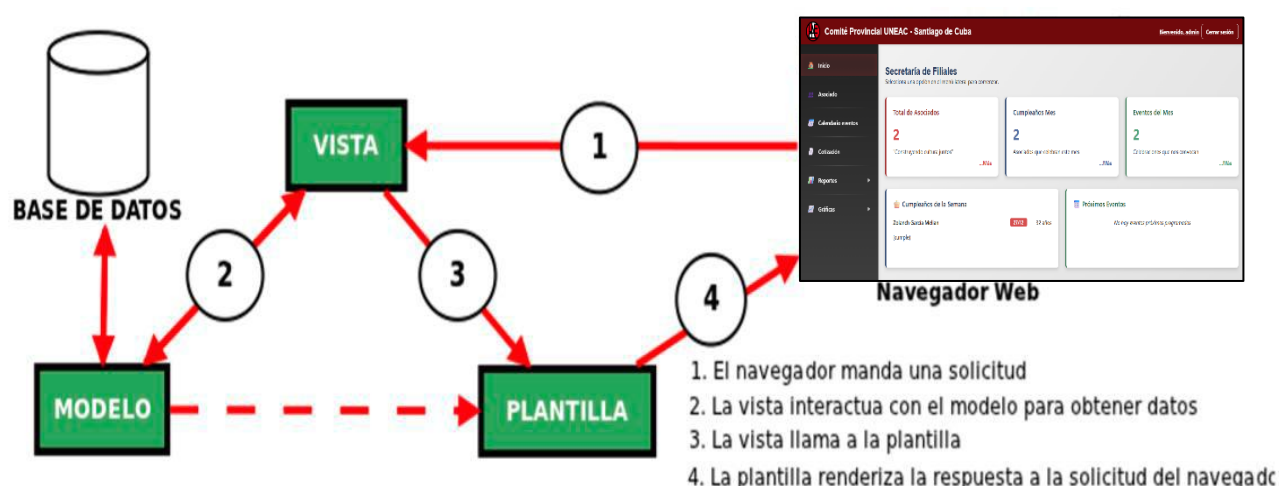


Figura 2. 4 Patrón de Arquitectura MTV

2.16 Conclusiones del capítulo

En el presente capítulo se abordaron las fases de planificación y diseño del desarrollo del software, destacando los elementos fundamentales que orientan la construcción de la solución. Se presentaron las funcionalidades a implementar conforme a los artefactos definidos por la metodología Extreme Programming (XP), entre los que sobresalen las historias de usuario y las historias técnicas, utilizadas para describir los requerimientos que el sistema debe satisfacer y establecer la prioridad de cada uno en la siguiente etapa de desarrollo. Asimismo, se elaboró un diseño de base de datos que garantiza el almacenamiento estructurado de toda la información necesaria para la gestión de la membresía del Comité Provincial de la UNEAC en Santiago de Cuba, asegurando la integridad y disponibilidad de los datos institucionales.

CAPÍTULO III IMPLEMENTACIÓN Y PRUEBAS

3.1 Introducción

El presente capítulo aborda la fase de implementación y pruebas del sistema, en esta etapa se materializa el diseño previamente definido, trasladando los modelos conceptuales y arquitectónicos a componentes de software funcionales, desarrollados en el entorno tecnológico seleccionado. La codificación se realizó empleando el lenguaje de programación Python y el framework Django, bajo el patrón de arquitectura MVT (Modelo–Vista–Template), lo que permitió mantener una clara separación de responsabilidades y además garantizando la coherencia entre la lógica de negocio, la presentación y la gestión de datos.

En correspondencia con la metodología ágil Extreme Programming (XP), la implementación y las pruebas se ejecutaron de manera paralela, favoreciendo la retroalimentación continua, la integración frecuente y la refactorización sistemática. Este enfoque permitió que cada funcionalidad se validara en el mismo ciclo de desarrollo, asegurando que el producto evolucionara de forma incremental y alineada con los requerimientos institucionales. Se documentan en este capítulo las decisiones tecnológicas adoptadas, la estructura de directorios del sistema, las medidas de seguridad y control de acceso implementadas, así como las validaciones realizadas para garantizar la integridad y confiabilidad de la información.

Finalmente, se presentan los casos de prueba y pruebas de aceptación aplicados al sistema, los cuales verificaron el cumplimiento de las historias de usuario y los requisitos funcionales y no funcionales definidos en etapas anteriores. De esta manera, este capítulo constituye la evidencia técnica de cómo la propuesta se transformó en una solución informática concreta, validada y lista para su despliegue en el entorno institucional.

3.2 ¿Cómo se aplica la filosofía de Django en la implementación del sistema?

La filosofía de **Django** se fundamenta en la organización clara y estructurada del código, lo que garantiza mantenibilidad, escalabilidad y coherencia en el desarrollo de aplicaciones web. Un proyecto en Django se compone de una serie de carpetas y archivos que siguen una estructura básica predefinida, dentro de la cual se ubican elementos esenciales como la configuración del sistema, la gestión de seguridad, el enrutamiento de las solicitudes y la definición de modelos, vistas y plantillas. Aunque esta estructura inicial es fija y obligatoria, el framework permite adaptaciones en función de las características particulares de cada aplicación.[35] De este modo, el desarrollador puede añadir módulos o directorios propios para extender funcionalidades y personalizar el sistema, sin perder la coherencia con las convenciones establecidas por el framework. Esta filosofía de trabajo, basada en la reutilización de componentes y en la claridad de responsabilidades, facilita la implementación de soluciones robustas y seguras, alineadas con las necesidades institucionales.

3.3 Estructura de archivos fundamentales en un proyecto Django

En el desarrollo de aplicaciones web con el framework Django, la organización modular del proyecto constituye un aspecto esencial para garantizar la mantenibilidad, escalabilidad y claridad del código. A continuación, se describen los principales archivos y directorios que conforman la base mínima de un proyecto Django, tomando como referencia la aplicación denominada `secretaria_filiales`.

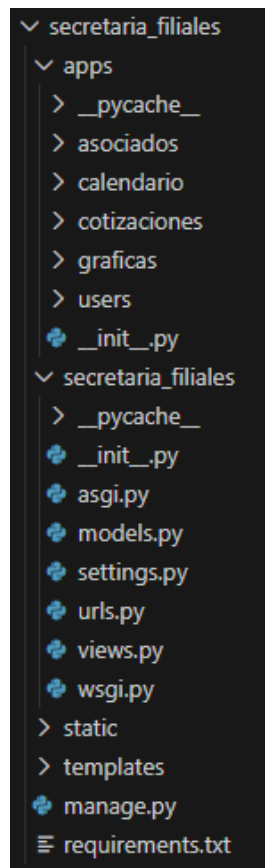


Figura 3. 1 Estructura general de una aplicación en Django

Directorios principales

- **apps/** Contiene las aplicaciones internas del proyecto. En este caso, se incluyen submódulos como *asociados*, *calendario*, *cotizaciones*, *graficas* y *users*, cada uno representando una unidad funcional del sistema. Este enfoque modular permite una separación clara de responsabilidades y facilita el mantenimiento.
- **secretaria_filiales/** (*directorio del núcleo del proyecto*) Este directorio contiene la configuración central y los componentes esenciales del proyecto Django:
- **__init__.py** Archivo vacío cuya función es declarar el directorio como un paquete de Python. Su inclusión habilita la importación de módulos dentro del contexto del proyecto, facilitando la modularidad y reutilización del código.
- **manage.py** Este archivo actúa como interfaz de línea de comandos para interactuar con el proyecto Django. Permite ejecutar tareas administrativas como migraciones, ejecución del servidor de desarrollo, pruebas unitarias,

entre otras. Su presencia es indispensable para la gestión operativa del entorno de desarrollo.

- **settings.py** Contiene la configuración global del proyecto, incluyendo parámetros como la base de datos, aplicaciones instaladas, rutas de archivos estáticos, middleware, y otros aspectos críticos para el funcionamiento del sistema.
- **urls.py** Define el sistema de enrutamiento de la aplicación. A través de expresiones regulares o funciones de path, se especifica qué vista debe ser invocada en respuesta a cada patrón de URL, estableciendo así la lógica de navegación del sitio.
- **wsgi.py** Implementa la interfaz WSGI (Web Server Gateway Interface), que constituye el estándar para la comunicación entre servidores web y aplicaciones Python. Este archivo es utilizado principalmente en entornos de producción para el despliegue del proyecto.
- **models.py** Define las estructuras de datos mediante clases que representan tablas en la base de datos. Cada clase modela una entidad del dominio, especificando sus atributos, relaciones y comportamientos asociados.
- **views.py** Contiene la lógica de presentación y procesamiento de las solicitudes HTTP. Las vistas determinan qué datos se renderizan y cómo se responden las peticiones del usuario, ya sea mediante funciones o clases basadas en vistas.
- **forms.py** (*creado posteriormente*) Este archivo se incorpora para definir formularios personalizados, facilitando la validación de datos y la interacción con el modelo desde la interfaz de usuario. Su uso es común en aplicaciones que requieren entrada estructurada por parte del usuario.
- Directorios auxiliares
- **static/** Almacena archivos estáticos como hojas de estilo (CSS), scripts JavaScript e imágenes. Estos recursos son servidos directamente al cliente sin procesamiento adicional.
- **templates/** Contiene las plantillas HTML utilizadas para renderizar las vistas. Permite separar la lógica de presentación del backend, siguiendo el patrón MVC (Modelo-Vista-Controlador).

3.4 Implementación del Sistema

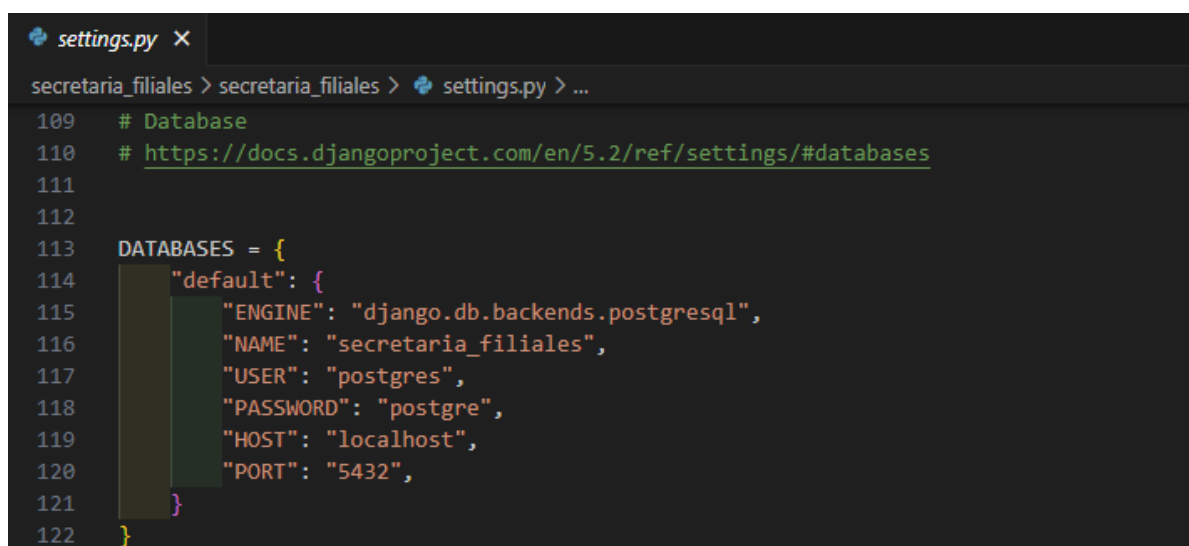
Presenta el proceso de construcción del sistema, desde la configuración inicial hasta la integración de los distintos componentes, en el mismo se mostrarán segmentos de código de ciertas funcionalidades del sistema, así como de la configuración realizada para lograr la correcta conexión entre el framework Django y la base de datos PostgreSQL.

3.4.1 Conectando la Base de Datos

Por defecto, el framework Django establece una conexión inicial con una base de datos SQLite, la cual se configura automáticamente en el archivo settings.py al momento de crear el proyecto. Esta opción resulta útil para entornos de desarrollo rápido o pruebas locales, pero no es recomendable para sistemas que requieren robustez, concurrencia y escalabilidad en producción.

Para este proyecto, se sustituyó la configuración predeterminada de SQLite por una conexión explícita con el sistema gestor de bases de datos PostgreSQL, ampliamente reconocido por su rendimiento, integridad transaccional y compatibilidad con Django.

La modificación se realizó en el bloque DATABASES del archivo settings.py, reemplazando el motor de base de datos y especificando los parámetros de conexión correspondientes:



```
settings.py X
secretaria_filiales > secretaria_filiales > settings.py > ...
109 # Database
110 # https://docs.djangoproject.com/en/5.2/ref/settings/#databases
111
112
113 DATABASES = {
114     "default": {
115         "ENGINE": "django.db.backends.postgresql",
116         "NAME": "secretaria_filiales",
117         "USER": "postgres",
118         "PASSWORD": "postgre",
119         "HOST": "localhost",
120         "PORT": "5432",
121     }
122 }
```

Figura 3. 2 Configuración del proyecto para conectar la base de datos

Descripción de los parámetros:

- **ENGINE:** Define el backend de base de datos utilizado. En este caso, se especifica `django.db.backends.postgresql` para habilitar la conexión con PostgreSQL.
- **NAME:** Nombre de la base de datos creada en el servidor PostgreSQL.
- **USER y PASSWORD:** Credenciales de acceso al motor de base de datos.
- **HOST:** Dirección del servidor donde se encuentra alojada la base de datos. Para entornos locales se utiliza `localhost`.
- **PORT:** Puerto de conexión estándar para PostgreSQL, generalmente 5432.

Esta configuración permite que el ORM de Django interactúe directamente con PostgreSQL, facilitando la creación, consulta y manipulación de datos mediante modelos definidos en `models.py`. Además, habilita el uso de migraciones automáticas, relaciones entre entidades y validaciones integradas, todo dentro del ecosistema Django.

3.4.2 Creando los modelos

Cada clase de modelo en Django representa una tabla en la base de datos; cada instancia de esa clase corresponde a una fila (registro) dentro de dicha tabla. En las siguientes figuras se muestran algunos de los modelos más relevantes en los que se detalla la definición de las clases que representan las tablas, sus atributos y relaciones (OneToMany y ManyToMany).

```

models.py x
secretaria_filiales > secretaria_filiales > models.py > ...
1 from django.db import models
2
3 class ModeloBase(models.Model):
4     creado_en = models.DateTimeField(
5         auto_now_add=True,
6         verbose_name="Fecha de creación"
7     )
8     actualizado_en = models.DateTimeField(
9         auto_now=True,
10        verbose_name="Última actualización"
11    )
12    activo = models.BooleanField(
13        default=True,
14        verbose_name="¿Está activo?"
15    )
16
17    def clean(self):
18        super().clean()
19
20
21    class Meta:
22        abstract = True
23        ordering = ["-creado_en"]
24

```

Figura 3. 3 Clase abstracta ModeloBase

Este fragmento de código define una clase base abstracta llamada ModeloBase, diseñada para ser heredada por otros modelos dentro del proyecto. Su propósito es centralizar atributos comunes que se repiten en múltiples entidades del sistema, promoviendo la reutilización de código y la coherencia estructural.

```

class Asociado(ModeloBase):
    class SexoChoices(models.TextChoices):
        MASCULINO = "M", "Masculino"
        FEMENINO = "F", "Femenino"

    class NivelEscolarChoices(models.TextChoices):
        PRIMARIO = "PR", "Primario"
        SECUNDARIO = "SE", "Secundario"
        OBRERO = "OC", "Obrero Calificado"
        TECNICO = "TP", "Técnico Profesional"
        SUPERIOR = "NS", "Nivel Superior"

    class OcupacionChoices(models.TextChoices):
        FUNCIONARIO = "FU", "Funcionario"
        TGP = "TC", "TGP"
        JUBILADO = "JU", "Jubilado"

    nombre = models.CharField(max_length=40, verbose_name="Nombre", blank=False)
    segundo_nombre = models.CharField(max_length=40, verbose_name="Segundo nombre", blank=True)
    apellido_paterno = models.CharField(max_length=40, verbose_name="Apellido paterno", blank=False)
    apellido_materno = models.CharField(max_length=40, verbose_name="Apellido materno", blank=True)
    fecha_nacimiento = models.DateField(verbose_name="Fecha de nacimiento", null=False, blank=False)
    sexo = models.CharField(max_length=1, choices=SexoChoices.choices, verbose_name="Sexo")
    carnet_identidad = models.CharField(max_length=11, verbose_name="Carnet de identidad", blank=False, unique=True)
    no_pasaporte = models.CharField(max_length=7, verbose_name="Número de pasaporte", blank=True)
    direccion_particular = models.CharField(max_length=255, verbose_name="Dirección particular", blank=True)
    email = models.EmailField(max_length=100, verbose_name="Correo electrónico", blank=True)
    nivel_escolar = models.CharField(max_length=2, choices=NivelEscolarChoices.choices, verbose_name="Nivel escolar", blank=False, default="PR",)
    ocupacion = models.CharField(max_length=2, choices=OcupacionChoices.choices, verbose_name="Ocupación", blank=False, default="FU",)
    profesion = models.CharField(max_length=20, verbose_name="Profesión", blank=True, help_text="Ejemplo: Actor, Pintor, Músico, Escritor",)
    no_miembro = models.CharField(max_length=10, verbose_name="Número de miembro", unique=True, blank=False, null=False, validators=[RegexValidator(r"^\d+$", "Solo se permiten números.")],
        help_text="Número de identificación como miembro de la UNEAC",)

    class Meta:
        verbose_name = "Asociado"
        verbose_name_plural = "Asociados"
        ordering = ["apellido_paterno", "apellido_materno", "nombre"]

```

Figura 3. 4 Clase Asociado

Este fragmento de código define la clase llamada Asociado. La cual constituye la representación de un miembro dentro del sistema institucional. La definición del modelo responde a la necesidad de mantener un registro ordenado y coherente de

los asociados, facilitando tanto la gestión administrativa como la consulta de información relevante en procesos institucionales.

```
class Membresia(ModelBase):
    CAUSALES_BAJA = [("traslado", "Traslado a otro Comité"), ("solicitud", "Solicitud Propia"), ("sancion", "Sanción de Separación Definitiva"), ("deceso", "Deceso"),]
    asociado = models.ForeignKey("Asociado", on_delete=models.CASCADE, related_name="membresias", verbose_name="Asociado",)
    filial = models.ForeignKey("Filial", on_delete=models.PROTECT, related_name="membresias", verbose_name="Filial",)
    ejecutivo_provincial = models.ForeignKey("EjecutivoProvincial", on_delete=models.SET_NULL, null=True, blank=True, verbose_name="Cargo provincial",)
    ejecutivo_filial = models.ForeignKey("EjecutivoFilial", on_delete=models.SET_NULL, null=True, blank=True, verbose_name="Cargo en filial",)
    comisiones = models.ManyToManyField("Comision", blank=True, verbose_name="Comisiones temáticas")
    distinciones = models.ManyToManyField("Distincion", blank=True, verbose_name="Distinciones recibidas")
    fecha_ingreso = models.DateField(verbose_name="Fecha de ingreso a la UNEAC")
    activo = models.BooleanField(default=True, verbose_name="¿Está activo en la UNEAC?")
    exento_cotizacion = models.BooleanField(default=False, verbose_name="¿Exento de pago de cotización?")
    fecha_baja = models.DateField(null=True, blank=True, verbose_name="Fecha de baja")
    causal_baja = models.CharField(max_length=20, choices=CAUSALES_BAJA, null=True, blank=True, verbose_name="Causal de baja",)

    class Meta:
        verbose_name = "Membresia"
        verbose_name_plural = "Membresias"
        ordering = ("asociado", "filial")
        unique_together = ("asociado", "filial")
```

Figura 3. 5 Clase Membresia

Este fragmento de código define la clase Membresia, la cual representa la afiliación de un asociado a una filial dentro de la organización. Esta arquitectura relacional contribuye a una gestión estructurada de la afiliación institucional, permitiendo reflejar tanto la dimensión administrativa como la participación activa de cada miembro en la organización.

3.4.3 La Vista

En el framework Django, la Vista constituye la capa controladora dentro del patrón de arquitectura MTV (Model-Template-View). Su función principal es determinar qué datos serán presentados al usuario, sin definir directamente cómo se mostrarán en la interfaz. Las vistas pueden implementarse como funciones o clases, y son responsables de gestionar la lógica de negocio asociada a cada operación.

En las siguientes figuras se muestran las vistas correspondientes a algunos de los modelos previamente definidos, donde se evidencia cómo cada vista interactúa con los modelos para ejecutar las operaciones de gestión y, posteriormente, transmite los resultados a las plantillas para su representación. Este diseño asegura una separación clara de responsabilidades y una mayor mantenibilidad del sistema.

```

# ELIMINAR ASOCIADO
@login_required
@permiso_requerido(lambda u: u.puede_ver_asociados())
def eliminar_asociado(request, pk):
    if request.method == "POST":
        asociado = get_object_or_404(Asociado, pk=pk)
        nombre_completo = asociado.nombre_completo
        asociado.delete()
        messages.success(request, f'Asociado "{nombre_completo}" eliminado correctamente.')
        return redirect("asociados:listar_asociados")
    # Si no es POST, redirigir a la lista
    messages.error(request, "Método no permitido.")
    return redirect("asociados:listar_asociados")

```

Figura 3. 6 Vista que permite Eliminar Asociados

Este fragmento de código define la vista `eliminar_asociado`, la cual permite suprimir del sistema el registro de un asociado previamente creado. Esta vista ejemplifica el principio de seguridad y control en operaciones críticas.

3.4.4 Las Plantillas

Las plantillas constituyen la capa de presentación dentro del patrón de arquitectura MTV (Model-Template-View) implementado por Django. Su función principal es definir cómo se visualizan los datos que han sido procesados por las vistas, permitiendo generar páginas HTML dinámicas a partir del contexto proporcionado por el controlador. Estas plantillas se estructuran mediante el lenguaje de marcado HTML enriquecido con etiquetas propias de Django, conocidas como etiquetas de plantilla y filtros, que permiten insertar variables, controlar estructuras condicionales, iterar sobre listas y reutilizar fragmentos de código mediante herencia.

En el sistema desarrollado, las plantillas se encargan de representar formularios, listados y otras interfaces de interacción con el usuario, manteniendo una separación clara entre la lógica de negocio y la presentación visual.

```

{% extends 'base.html' %} {% load static %}{% load asociados_tags %} {% load filters %}
{% block extra_css %}
<link rel="stylesheet" href="{% static 'css/index.css' %}" />
{% endblock %}
{% block content %}
<div class="separador-md"></div>
<div class="encabezado-seccion">
<h2>Secretaría de Filiales</h2>
<p class="intro-text">Selecciona una opción en el menú lateral para comenzar.</p>
</div>
<div class="separador-md"></div>
<div class="dashboard-cards">
<div class="card card-asociado">
<h3>Total de Asociados</h3>
<div class="numero">{% total_asociados %}</div>
<p>"Construyendo cultura juntos"</p>
<a href="{% url 'asociados:reporte_asociados_alfabetico' %}" class="btn-ver-mas">...Más</a>
</div>
<div class="card card-cumple">
<h3>Cumpleaños Mes</h3>
<div class="numero">{% cumpleaños_mes %}</div>
<p>Asociados que celebran este mes</p>
<a href="{% url 'asociados:reporte_cumpleanios' %}" class="btn-ver-mas">...Más</a>
</div>
<div class="card card-eventos">
<h3>Eventos del Mes</h3>
<div class="numero">{% eventos_mes %}</div>
<p>Celebraciones que nos convocan</p>
<a href="{% url 'calendario:reporte_eventos' %}" class="btn-ver-mas">...Más</a>
</div>
</div>
<div class="dashboard-cards-grandes">
{% card_cumpleanios_semana %}
{% card_proximos_eventos %}
</div>
{% block extra_js %}
<script src="{% static 'js/chart.min.js' %}"></script>
<script src="{% static 'js/grafico_filiales.js' %}"></script>
<script src="{% static 'js/grafico_filiales_barras.js' %}"></script>
<script src="{% static 'js/chartjs-plugin-datalabels.min.js' %}"></script>
{% endblock %}
{% endblock %}

```

Figura 3. 7 Plantilla Index

Este fragmento de código define la plantilla index.html, utilizada como panel principal de navegación del sistema. Hereda de la plantilla base y organiza la presentación de indicadores clave mediante tarjetas informativas, bloques reutilizables y etiquetas personalizadas. Su propósito es ofrecer una vista sintética del estado institucional, facilitando el acceso a reportes y secciones relevantes.

3.4.5 Las URLs

En el framework Django, el sistema de URLs constituye el mecanismo de enrutamiento que permite vincular las solicitudes del navegador con las vistas correspondientes. Esta capa actúa como intermediaria entre la interfaz del usuario y la lógica de negocio, definiendo qué función o clase controladora debe ejecutarse ante una determinada dirección web. En las siguientes figuras se presentan ejemplos representativos de archivos urls.py.

```

from django.urls import path, include
from django.contrib import admin
from django.contrib.auth.views import LogoutView
from . import views

urlpatterns = [
    path("admin/", admin.site.urls),
    path('', views.CustomLoginView.as_view(), name='login'),
    path('login/', views.CustomLoginView.as_view(), name='login'),
    path('logout/', LogoutView.as_view(next_page='login'), name='logout'),
    path('index/', views.index, name='index'),
    path("asociados/", include(("apps.asociados.urls", "asociados"), namespace="asociados")),
    path("cotizaciones/", include(("apps.cotizaciones.urls", "cotizaciones"), namespace="cotizaciones")),
    path("calendario/", include(("apps.calendario.urls", "calendario"), namespace="calendario")),
    path("graficas/", include(("apps.graficas.urls", namespace="graficas")),
    path("users/", include(("apps.users.urls", namespace="users")),
]

```

Figura 3. 8 URL principal del proyecto.

Este fragmento de código define el archivo urls.py del proyecto Django, el cual centraliza la configuración de rutas y coordina el acceso a los distintos módulos de la aplicación. Su estructura permite organizar el sistema en aplicaciones independientes, cada una con su propio espacio de nombres, lo que favorece la modularidad, la escalabilidad y el mantenimiento del código. Esta configuración constituye el núcleo del sistema de enrutamiento, permitiendo que cada solicitud HTTP sea dirigida a la vista correspondiente de forma clara y estructurada. Cada ruta está asociada a un nombre único nombre, lo que permite generar enlaces reversibles desde las plantillas.

```

from django.urls import path
from . import views

app_name = "asociados"

urlpatterns = [
    path("", views.listar_asociados, name="listar_asociados"),
    path("gestionar/", views.gestionar_asociado, name="crear_asociado"),
    path("gestionar/<int:pk>/", views.gestionar_asociado, name="editar_asociado"),
    path("eliminar/<int:pk>/", views.eliminar_asociado, name="eliminar_asociado"),
    path("reportes/reportes_asociados_alfabetico/", views.reporte_asociados_alfabetico, name="reporte_asociados_alfabetico"),
    path("reportes/ejecutivo-provincial/", views.reporte_ejecutivo_provincial, name="reporte_ejecutivo_provincial"),
    path("reportes/ejecutivo-filial/", views.reporte_ejecutivo_filial, name="reporte_ejecutivo_filial"),
    path("reportes/cumpleaños/", views.reporte_cumpleaños, name="reporte_cumpleaños"),
    path("reportes/premios-distinciones/", views.reporte_premios_distinciones, name="reporte_premios_distinciones"),
    path("reportes/comisiones-trabajo/", views.reporte_comisiones_trabajo, name="reporte_comisiones_trabajo"),
    path("reportes/reportes-baja/", views.reporte_asociados_baja, name="reporte_asociados_baja"),
]

```

Figura 3. 9 URL módulo de asociados.

Este fragmento de código define el archivo urls.py correspondiente al módulo asociados, el cual organiza las rutas internas que permiten gestionar las operaciones vinculadas al modelo Asociado.

3.5 Paquetes Utilizados

El sistema desarrollado se apoya en un conjunto de paquetes internos de Django y bibliotecas externas que amplían sus funcionalidades, garantizando la seguridad, la

eficiencia y la presentación institucional de la aplicación. A continuación, se describen los principales paquetes utilizados:

- **django.contrib.auth**: módulo nativo de Django encargado de la autenticación y autorización de usuarios.
- **django.contrib.messages**: proporciona un sistema de mensajería temporal para mostrar notificaciones de éxito, advertencia o error en operaciones como creación, edición o eliminación de registros.
- **psycpg2-binary**: controlador de base de datos PostgreSQL para Python, utilizado como puente entre Django y el motor de base de datos, garantizando persistencia y eficiencia en las consultas.
- **sqlparse**: empleado por Django para el manejo y formateo de consultas SQL.
- **Chart.js**: biblioteca JavaScript para la generación de gráficos interactivos en el módulo de graficas.
- **weasyprint**: convierte plantillas HTML y estilos CSS en documentos PDF, fundamentales para la producción de reportes oficiales.
- **pillow**: biblioteca de procesamiento de imágenes que complementa a WeasyPrint, permitiendo la inclusión y manipulación de logotipos y gráficos en los documentos PDF.

La integración de estos paquetes responde a criterios de estabilidad, compatibilidad y documentación, asegurando que el sistema sea modular, escalable y adaptable a los estándares institucionales. Mientras los módulos internos de Django garantizan la seguridad y la gestión básica, las bibliotecas externas aportan capacidades avanzadas de presentación, generación de reportes y visualización de datos.

3.6 Manejo de la seguridad del sistema

La seguridad en aplicaciones web constituye un proceso fundamental que se desarrolla en dos fases complementarias: **autenticación** y **autorización**. Su objetivo es garantizar que los usuarios accedan únicamente a los recursos que les corresponden, evitando vulnerabilidades y accesos indebidos.

Autenticación: es el primer paso del proceso de seguridad. El sistema solicita al usuario algún tipo de identificación (generalmente credenciales como nombre de usuario y contraseña) y valida su identidad. En Django, este proceso se gestiona

mediante el módulo `django.contrib.auth`, que provee mecanismos integrados para el inicio de sesión, cierre de sesión y recuperación de credenciales.

Autorización: una vez autenticado el usuario, el sistema determina si posee los permisos necesarios para acceder al recurso solicitado. Django implementa este control mediante permisos asociados a modelos y grupos, lo que permite definir roles institucionales y restringir acciones específicas como creación, edición o eliminación de registros.

3.6.1 Implementación del Sistema de Autorización Personalizado

Para satisfacer los requisitos específicos del sistema de gestión de asociados, se implementó un sistema de seguridad de dos capas: una capa estándar de Django, responsable de la autenticación básica, protección CSRF, administración de sesiones y mensajería además de componentes nativos del framework, y una capa personalizada, que implementa permisos específicos del negocio, auditoría automatizada y control de acceso granular. Este sistema de autorización personalizado complementa los mecanismos estándar de Django y se estructura en cuatro componentes principales:

- Procesador de Contexto (`context_processors.py`): Inyecta los permisos del usuario en todas las plantillas, permitiendo una interfaz adaptativa.
- Decoradores de Control (`decorators.py`): Validan permisos y roles antes de ejecutar las vistas basadas en funciones.
- Middleware de Auditoría (`middleware.py`): Registra automáticamente los accesos de los usuarios, incluyendo marca temporal y dirección IP.
- Mixins para Vistas (`mixins.py`): Extienden el control de acceso a las vistas basadas en clases.

Este enfoque multinivel garantiza el principio de mínimo privilegio, combinando las robustas protecciones nativas de Django con controles específicos del dominio, reduciendo la superficie de ataque y mejorando la seguridad general del sistema.

```
<form method="post" class="login-form">
  {% csrf_token %}
  <input type="text" name="username" placeholder="Usuario" required autocomplete="username" />
  <input type="password" name="password" placeholder="Contraseña" required autocomplete="current-password" />
  <button type="submit" class="login-btn">Entrar</button>
</form>
```

Figura 3. 10 CSRF_TOKEN en formulario de login.

CSRF (Cross-Site Request Forgery): Django genera automáticamente un *token* CSRF para cada sesión activa. Este token se incluye en los formularios y solicitudes POST, verificando que la acción proviene del usuario autenticado y no de una aplicación externa malintencionada. Al regenerarse en cada sesión, se impide que terceros puedan reutilizarlo.

```
def rol_requerido(*roles_requeridos):
    def decorator(view_func):
        @wraps(view_func)
        def _wrapped_view(request, *args, **kwargs):
            if not request.user.is_authenticated:
                return redirect('login')

            if request.user.role not in roles_requeridos:
                from django.contrib import messages
                messages.error(request, "No tienes permiso para acceder a esta página.")
                return redirect('index')
            return view_func(request, *args, **kwargs)
        return _wrapped_view
    return decorator
```

Figura 3. 11 Control de acceso basado en roles (decorators.py)

Control de acceso basado en roles: Se implementó el decorador `@rol_requerido` que valida el rol del usuario antes de ejecutar cualquier vista protegida. Este mecanismo garantiza que cada funcionalidad sea accesible únicamente por los perfiles institucionales correspondientes.

Gestión de sesiones: las credenciales y tokens se almacenan en la sesión del usuario, la cual se administra de forma segura por Django. Esto garantiza que la información sensible no sea accesible desde aplicaciones externas.

Mensajería y retroalimentación: El sistema informa al usuario sobre el resultado de sus acciones (éxito, error, advertencia), evitando confusión y reforzando la transparencia en los procesos de seguridad.

3.7 Pruebas realizadas al sistema

Las pruebas de software constituyen un conjunto de actividades destinadas a identificar posibles fallos en el funcionamiento de la aplicación y a verificar que los requisitos definidos se cumplen de manera satisfactoria. Un caso de prueba se compone de condiciones o variables bajo las cuales el analista determina si el sistema responde parcial o completamente de acuerdo con lo esperado.

La metodología adoptada divide las pruebas en tres categorías: pruebas unitarias, de funcionalidad y de aceptación. El cliente participa activamente en este proceso, especificando uno o varios escenarios para comprobar la correcta implementación de cada historia de usuario y verificando que los resultados obtenidos coincidan con los esperados.

Durante la implementación de la aplicación se diseñó un conjunto de casos de prueba para comprobar su funcionamiento integral, en correspondencia con los requerimientos descritos en las historias de usuario del capítulo anterior. A continuación, se presentan algunas de las pruebas de funcionalidad y de aceptación realizadas.

Tabla 3. 1 Prueba de Aceptación: Autenticar Usuarios

Caso de Prueba de aceptación	
Código de caso de Prueba: CPH_1	Número de la Historia de Usuario: HU_1
Descripción de la Prueba: Verificar que el sistema autentique a los usuarios con credenciales válidas.	
Condiciones de Ejecución: Las credenciales deben estar registradas en la base de datos.	
Entrada/Pasos de Ejecución: 1. Acceder al sistema. 2. Ingresar usuario y contraseña en el formulario de autenticación. 3. Presionar el botón "Entrar".	
Resultado de la Prueba: - Si las credenciales son correctas: acceso a la página de inicio correspondiente al rol del usuario. - Si las credenciales son incorrectas: mostrar mensaje de error y denegar el acceso.	
Evaluación de la Prueba: El sistema respondió conforme a lo esperado. Prueba satisfactoria.	

Tabla 3. 2 Prueba de Aceptación: Registrar Asociado

Caso de Prueba de aceptación	
Código de caso de Prueba: CPH_2	Número de la Historia de Usuario: HU_2
Descripción de la Prueba: Verificar que el sistema permita registrar correctamente un nuevo asociado ingresando todos los campos obligatorios.	
Condiciones de Ejecución: El usuario debe tener permisos de Administrador o Secretario.	
Entrada/Pasos de Ejecución: 1. Acceder al módulo de asociados. 2. Presionar el botón "Crear Asociado". 3. Completar el formulario con los datos requeridos. 4. Presionar el botón "Crear Asociado".	
Resultado de la Prueba: - Si todos los campos obligatorios están completos y válidos: el sistema guarda el nuevo asociado en la base de datos y muestra un mensaje de confirmación. - Si falta algún campo obligatorio o hay datos inválidos: el sistema muestra un mensaje de error indicando qué información debe corregirse.	
Evaluación de la Prueba: El sistema respondió conforme a lo esperado. Prueba satisfactoria.	

Tabla 3. 3 Prueba de Aceptación: Registrar Cotización

Caso de Prueba de aceptación	
Código de caso de Prueba: CPH_3	Número de la Historia de Usuario: HU_3
Descripción de la Prueba: Verificar que el sistema permita registrar una cotización válida asociada a un miembro existente.	
Condiciones de Ejecución:	

- El usuario debe tener permisos de Administrador o Secretario. - El asociado debe estar registrado previamente en el sistema.
Entrada/Pasos de Ejecución: 1. Acceder al módulo de cotizaciones. 2. Seleccionar el botón "Crear comprobante". 3. Completar el formulario con los datos requeridos. 4. Presionar el botón "Crear comprobante" para guardar.
Resultado de la Prueba: - Si los datos son válidos y el asociado existe: la cotización se guarda en la base de datos y aparece en el listado de cotizaciones con confirmación de registro. - Si los datos son incompletos o el asociado no existe: el sistema muestra un mensaje de error indicando la causa y no guarda la cotización.
Evaluación de la Prueba: El sistema respondió conforme a lo esperado. Prueba satisfactoria.

Tabla 3. 4 Prueba de Aceptación: Crear Usuario del Sistema

Caso de Prueba de aceptación	
Código de caso de Prueba: CPH_4	Número de la Historia de Usuario: HU_5
Descripción de la Prueba: Verificar que el sistema permita crear un nuevo usuario con un rol asignado.	
Condiciones de Ejecución: - El usuario debe tener permisos de Administrador.	
Entrada/Pasos de Ejecución: 1. Acceder al módulo de administración. 2. Dar clic en el botón "Agregar usuario". 3. Completar el formulario con los datos requeridos. 4. Dar clic en el botón "Guardar".	
Resultado de la Prueba: - Si los datos son válidos: el nuevo usuario queda registrado en la base de datos y puede autenticarse. - Si los datos son incompletos o inválidos: el sistema muestra un mensaje de error indicando la causa y no guarda el usuario.	

Evaluación de la Prueba:

El sistema respondió conforme a lo esperado. Prueba satisfactoria.

3.8 Validación

En las ciencias de la computación, la validación de datos constituye el proceso mediante el cual se asegura que un programa funcione sobre información limpia, correcta y útil. Este procedimiento se implementa a través de rutinas conocidas como reglas de validación, restricciones de validación o rutinas de comprobación, que verifican la corrección, significación y seguridad de los datos introducidos en el sistema antes de su procesamiento. La validación garantiza que los datos sean aptos para el propósito deseado, válidos, razonables y seguros.

En el sistema desarrollado para la UNEAC, la validación se implementó mediante un enfoque de doble capa que aprovecha las capacidades nativas de Django y complementa con reglas de negocio específicas, tanto a nivel de formularios como de modelos.

3.8.1 Validación en formularios

Los formularios de Django constituyen la primera línea de defensa en la validación de datos. En el sistema se implementaron validaciones que combinan:

- Validaciones básicas de Django: Campos requeridos, longitud máxima, formatos específicos (email, fechas)
- Validaciones personalizadas de negocio:

```
def clean_carnet_identidad(self):
    carnet = self.cleaned_data["carnet_identidad"]
    if not carnet.isdigit() or len(carnet) != 11:
        raise ValidationError("El carnet de identidad debe contener exactamente 11 dígitos.")
    queryset = Asociado.objects.filter(carnet_identidad=carnet)
    if self.instance.pk:
        queryset = queryset.exclude(pk=self.instance.pk)
    if queryset.exists():
        raise ValidationError("Este carnet de identidad ya está registrado.")
    return carnet
```

Figura 3. 12 Validación de Carnet de Identidad en formulario

3.8.2 Validación en modelos

Cada modelo incorpora validaciones que garantizan la integridad referencial y reglas de negocio complejas:

- Validaciones de integridad: Campos únicos (carnet_identidad, no_miembro), relaciones ForeignKey
- Reglas de negocio complejas en Membresia.clean():

```
fecha_nacimiento = models.DateField(verbose_name="Fecha de nacimiento", null=False, blank=False)
sexo = models.CharField(max_length=1, choices=SexoChoices.choices, verbose_name="Sexo")
carnet_identidad = models.CharField(max_length=11, verbose_name="Carnet de identidad", blank=False, unique=True)
no_pasaporte = models.CharField(max_length=7, verbose_name="Número de pasaporte", blank=True)
direccion_particular = models.CharField(max_length=255, verbose_name="Dirección particular", blank=True)
email = models.EmailField(max_length=100, verbose_name="Correo electrónico", blank=True)
```

Figura 3. 13 Validaciones en el Modelo de Asociados.

```
def clean(self):
    errors = {}
    # Validación 1: No puede tener ambos cargos
    if self.ejecutivo_provincial and self.ejecutivo_filial:
        mensaje_error = "Un asociado no puede tener cargo en el ejecutivo provincial y en el ejecutivo filial al mismo tiempo."
        errors["ejecutivo_filial"] = mensaje_error
        errors["ejecutivo_provincial"] = mensaje_error
    # Validación 2: Si hay fecha_baja, debe haber causal_baja
    if self.fecha_baja and not self.causal_baja:
        errors["causal_baja"] = ("Debe especificar la causal de baja cuando hay fecha de baja.")
```

Figura 3. 14 Validaciones en el Modelo de Membresia

El método clean() implementa una validación integral sobre la instancia de membresía, asegurando la coherencia entre cargos, fechas, estado activo y límites reglamentarios.

3.8.3 Reglas de Negocio Específicas de la UNEAC

El sistema implementa validaciones específicas para los procesos institucionales:

Límites estructurales:

- Máximo 5 filiales en el sistema (Filial.clean())
- Máximo 4 cargos provinciales (EjecutivoProvincial.clean())
- Máximo 2 membresías activas por asociado

Reglas de membresía:

- Coherencia entre fecha y causal de baja
- En doble membresía, solo una puede estar exenta de cotización
- Validación de fechas (ingreso posterior a nacimiento)

Reglas de cargos:

- Límites por tipo de cargo (1 Presidente, 1 Vicepresidente por filial)
- Incompatibilidad entre cargos provinciales y filiales

Este enfoque de validación en múltiples niveles (formulario + modelo + base de datos) garantiza la integridad de los datos, la consistencia del sistema y el cumplimiento estricto de los procesos de la UNEAC, asegurando que toda información almacenada sea confiable y válida para la toma de decisiones administrativas.

3.9 Conclusiones del Capítulo

Este capítulo detalló la implementación del sistema en Django, presentando su arquitectura modular y la integración de modelos, vistas, plantillas y rutas para garantizar mantenibilidad. Se configuró la conexión con PostgreSQL y se implementaron validaciones en formularios y modelos para asegurar la integridad de los datos. El sistema incorpora un mecanismo de seguridad de dos capas (nativa de Django y personalizada) que aplica el principio de mínimo privilegio.

Las herramientas y paquetes utilizados extendieron las funcionalidades del sistema, y las pruebas realizadas confirmaron el cumplimiento de todos los requisitos. Los resultados obtenidos demuestran que el sistema es confiable, eficiente y pertinente para el entorno institucional de la UNEAC.

CONCLUSIONES GENERALES

En el desarrollo de esta investigación se estudió el estado del arte de los sistemas de gestión de asociados en el ámbito nacional e internacional, realizando las adaptaciones pertinentes para su implementación en el sistema propuesto para el Comité Provincial de la UNEAC en Santiago de Cuba; se seleccionaron las herramientas, metodologías y tecnologías más adecuadas, garantizando el cumplimiento de las exigencias institucionales y las políticas de software libre del país; se diseñó y confeccionó una base de datos relacional que abarca toda la información necesaria e implícita en la gestión de asociados, incluyendo datos personales, cotizaciones, eventos y roles administrativos; y finalmente se desarrolló y validó el funcionamiento de una aplicación web accesible y segura, que permite disminuir errores y pérdidas de información, incrementar la seguridad en el control de los datos, mejorar la trazabilidad de los procesos y optimizar el uso de recursos administrativos, consolidando así una solución tecnológica eficiente, confiable y alineada con los objetivos institucionales de la UNEAC.

La investigación valida la hipótesis planteada: la implementación de una solución tecnológica orientada a la automatización de los procesos de gestión de asociados incrementa la eficiencia operativa, reduce errores manuales y fortalece la toma de decisiones institucionales. El sistema desarrollado constituye un aporte práctico para el Comité Provincial de la UNEAC de Santiago de Cuba, al modernizar sus procesos administrativos y sentar las bases para futuras extensiones hacia los otros Comité Provinciales del país.

RECOMENDACIONES

A partir de los resultados obtenidos en la investigación y la implementación del sistema de gestión de asociados para el Comité Provincial de la UNEAC en Santiago de Cuba, se recomienda continuar con la evolución de la solución mediante la incorporación de nuevas funcionalidades que respondan a las necesidades emergentes de la organización, entre ellas el envío automatizado de correos personalizados a la membresía en las diversas áreas de gestión; la capacitación sistemática del personal encargado de operar la aplicación, con el fin de garantizar un uso eficiente y reducir la resistencia al cambio tecnológico; y la evaluación de la posibilidad de extender la solución a otros Comités Provinciales de la UNEAC en el país, consolidando así un modelo de gestión informatizado que contribuya a la modernización, seguridad y eficiencia de los procesos administrativos de la organización.

REFERENCIAS BIBLIOGRÁFICAS

- [1] «Una guía esencial para los sistemas de gestión de membresías para organizaciones». Accedido: 4 de julio de 2024. [En línea]. Disponible en: <https://www.affmu.com/es/an-essential-guide-to-membership-management-systems-for-organizations.html>
- [2] Z. Adams, «A History of Management», en *The Legal Concept of Work*, 1.^a ed., Oxford University Press Oxford, 2022, pp. 107-138. doi: 10.1093/oso/9780192857774.003.0006.
- [3] R. Zhovnovach, T. Petlenko, y A. Orlova, «Business management technologies in the management system», *Ukr. J. Appl. Econ. Technol.*, vol. 9, n.º 1, pp. 18-23, ene. 2024, doi: 10.36887/2415-8453-2024-1-2.
- [4] «Membree: Opiniones, precios & funcionalidades | Appvizer», appvizer.es. Accedido: 5 de julio de 2024. [En línea]. Disponible en: <https://www.appvizer.es/relacion-cliente/gestion-miembros/membree>
- [5] «Member Manager: Opiniones, precios & funcionalidades | Appvizer», appvizer.es. Accedido: 5 de julio de 2024. [En línea]. Disponible en: <https://www.appvizer.es/relacion-cliente/gestion-miembros/member>
- [6] «MemberLeap: Opiniones, precios & funcionalidades | Appvizer». Accedido: 5 de julio de 2024. [En línea]. Disponible en: <https://www.appvizer.es/relacion-cliente/gestion-miembros/memberleap>
- [7] «MemberPress: Opiniones, precios & funcionalidades | Appvizer», appvizer.es. Accedido: 5 de julio de 2024. [En línea]. Disponible en: <https://www.appvizer.es/relacion-cliente/gestion-miembros/memberpress>
- [8] «Daxko Operations: Opiniones, precios & funcionalidades | Appvizer», appvizer.es. Accedido: 5 de julio de 2024. [En línea]. Disponible en: <https://www.appvizer.es/relacion-cliente/gestion-miembros/daxko-operations>
- [9] «Definición de Sistema informático (SI)», Alegsa.com.ar. Accedido: 5 de julio de 2024. [En línea]. Disponible en: https://www.alegsa.com.ar/Dic/sistema_informatico.php
- [10] S. Lujan Mora, *PROGRAMACION DE APLICACIONES WEB*, 1.^a ed. Editorial Club Universitario, 2002.
- [11] «Definición de gestión - Qué es, Concepto y Significado.» Accedido: 5 de julio de 2024. [En línea]. Disponible en: <https://www.definicion.co/gestion/>
- [12] «¿Qué es un Sistema de Gestión? Definición, tipos y ejemplos», Estudiando. Accedido: 4 de julio de 2024. [En línea]. Disponible en: <https://estudiando.com/sistemas-de-gestion-definicion-tipos-y-ejemplos/>
- [13] «Naturaleza y concepto de la organización».
- [14] uninotas, «Organizaciones y Marco Legal: Guía Completa», UniNotas. Accedido: 7 de enero de 2026. [En línea]. Disponible en: <https://www.uninotas.net/organizaciones-y-marco-legal-guia-completa/>
- [15] J. Añez, «Entidad Legal: Definición, Qué es y Ejemplos | 2026». Accedido: 7 de enero de 2026. [En línea]. Disponible en: <https://www.economia360.org/entidad-legal/>
- [16] F. B. Pardo y H. Saber, «ASAMBLEA NACIONAL DEL PODER POPULAR».
- [17] «Filial», *Wikipedia, la enciclopedia libre*. 14 de abril de 2025. Accedido: 7 de enero de 2026. [En línea]. Disponible en: <https://es.wikipedia.org/w/index.php?title=Filial&oldid=166848039>

- [18] «¿Qué es ser un asociado? - Revista FP». Accedido: 7 de enero de 2026. [En línea]. Disponible en: <https://cifpvalentinpazandrade.es/que-es-ser-un-asociado/>
- [19] R.- ASALE y RAE, «membresía | Diccionario de la lengua española», «Diccionario de la lengua española» - Edición del Tricentenario. Accedido: 7 de enero de 2026. [En línea]. Disponible en: <https://dle.rae.es/membresia>
- [20] «Django overview», Django Project. Accedido: 7 de enero de 2026. [En línea]. Disponible en: <https://www.djangoproject.com/start/overview/>
- [21] «Visual Studio Code». Accedido: 7 de enero de 2026. [En línea]. Disponible en: https://www.filepuma.com/es/download/visual_studio_code_64bit_1.105.0-54490/
- [22] M. G. Ginestà y O. P. Mora, «Bases de datos en PostgreSQL», *SIJ: [sn]*, 2012.
- [23] «pgAdmin 4 — pgAdmin 4 8.9 documentation». Accedido: 24 de julio de 2024. [En línea]. Disponible en: <https://www.pgadmin.org/docs/pgadmin4/8.9/index.html>
- [24] «Visual Paradigm - EcuRed». Accedido: 7 de enero de 2026. [En línea]. Disponible en: https://www.ecured.cu/Visual_Paradigm
- [25] «Waitress — waitress 3.0.2 documentation». Accedido: 7 de enero de 2026. [En línea]. Disponible en: <https://docs.pylonsproject.org/projects/waitress/en/latest/index.html>
- [26] A. Raeburn, «¿Qué es la programación extrema (XP)? [2024]», Asana. Accedido: 16 de julio de 2024. [En línea]. Disponible en: <https://asana.com/es/resources/extreme-programming-xp>
- [27] «Python», Python documentation. Accedido: 7 de enero de 2026. [En línea]. Disponible en: <https://docs.python.org/3/tutorial/index.html>
- [28] «¿Qué es SQL? Características y usos de este lenguaje», UNIR. Accedido: 23 de enero de 2025. [En línea]. Disponible en: <https://www.unir.net/revista/marketing-comunicacion/que-es-sql/>
- [29] «HTML: Lenguaje de etiquetas de hipertexto | MDN», MDN Web Docs. Accedido: 7 de enero de 2026. [En línea]. Disponible en: <https://developer.mozilla.org/es/docs/Web/HTML>
- [30] «JavaScript | MDN», MDN Web Docs. Accedido: 7 de enero de 2026. [En línea]. Disponible en: <https://developer.mozilla.org/es/docs/Web/JavaScript>
- [31] «CSS | MDN», MDN Web Docs. Accedido: 7 de enero de 2026. [En línea]. Disponible en: <https://developer.mozilla.org/es/docs/Web/CSS>
- [32] «Qué es el lenguaje unificado de modelado (UML)», Lucidchart. Accedido: 7 de enero de 2026. [En línea]. Disponible en: <https://www.lucidchart.com/pages/es/que-es-el-lenguaje-unificado-de-modelado-uml>
- [33] «Actores Proyecto Informático: Roles Y Responsabilidades - ServerNet». Accedido: 7 de enero de 2026. [En línea]. Disponible en: <https://servernet.com.ar/actores-del-proyecto-informatico/>
- [34] M. Cohn, *User stories applied: for agile software development*, 18. print. en Addison-Wesley signature series. Boston, Mass.: Addison-Wesley, 2013.
- [35] W. S. Vincent, *Django for professionals: production websites with Python & Django*. United States? William S. Vincent, 2022.
- [36] *Ingeniería del software: un enfoque práctico*, Séptima edición. México; Bogotá; Buenos Aires; Caracas; Guatemala; Madrid: McGraw-Hill Education, 2013.
- [37] C. Coronel y S. Morris, *Database systems: design, implementation, and management*, Thirteenth edition. Mason, OH: Cengage Learning US, 2018.

- [38] C. Larman, *Applying UML and patterns: an introduction to object-oriented analysis and design and the unified process*, 2nd ed. en Safari electronic books. Upper Saddle River, NJ: Prentice Hall PTR, 2002.

ANEXOS

Tabla A- 1 Prueba unitaria: Validación de credenciales correctas

Caso de Prueba Unitaria	
Código de caso de Prueba: PU_Login_1	Módulo: secretaria_filiales
Objetivo: Verificar que el método form_valid() permita el acceso cuando las credenciales son válidas.	
Entradas: - Usuario: "prueba" - Contraseña: "password123"	
Condiciones de Ejecución: - Usuario registrado en la base de datos con credenciales válidas.	
Entrada/Pasos de Ejecución: 1. Invocar form_valid() con un formulario válido. 2. Capturar la respuesta.	
Resultado de la Prueba: - Redirección a '/index/'. - Actualización de registrar_acceso() con fecha/hora e IP. - Sesión activa durante 5 minutos.	
Evaluación de la Prueba: - El resultado coincide con lo esperado.	

Tabla A- 2 Prueba unitaria: Validación de credenciales incorrectas

Caso de Prueba Unitaria	
Código de caso de Prueba: PU_Login_2	Módulo: secretaria_filiales
Objetivo: Verificar que el método form_valid() no permita el acceso cuando las credenciales son válidas.	
Entradas: - Usuario: "prueba"	

- Contraseña: "clave incorrecta"
Condiciones de Ejecución: - Usuario registrado en la base de datos, pero con contraseña incorrecta.
Entrada/Pasos de Ejecución: 1. Invocar form_valid() con un formulario inválido. 2. Capturar la respuesta.
Resultado de la Prueba: - Renderización de login.html. - Modal de error visible en el template. - Retorno HTTP 200 con formulario y errores.
Evaluación de la Prueba: - El resultado coincide con lo esperado.

Tabla A- 3 Prueba de Funcionalidad 1: Inicio de sesión exitoso

Nombre de la Prueba: Inicio de sesión exitoso	
Código de caso de Prueba: PF_Login_1	Módulo: secretaria_filiales
Objetivo: Verificar que el sistema permita iniciar sesión con credenciales válidas.	
Condiciones de Ejecución: - Usuario registrado con credenciales correctas.	
Pasos de Ejecución: 1. Acceder al formulario de login. 2. Ingresar usuario y contraseña válidos. 3. Presionar "Entrar".	
Resultado esperado: - Redirección a '/index/'. - Sesión activa durante 5 minutos. - Registro de acceso con fecha/hora e IP.	
Evaluación de la Prueba: - El sistema respondió conforme a lo esperado.	

Tabla A- 4 Prueba de Funcionalidad 2: Inicio de sesión fallido

Nombre de la Prueba: Inicio de sesión fallido	
Código de caso de Prueba: PF_Login_2	Módulo: secretaria_filiales
Objetivo: Verificar que el sistema no permita iniciar sesión con credenciales inválidas.	
Condiciones de Ejecución: - Usuario registrado, pero con contraseña incorrecta.	
Pasos de Ejecución: 1. Acceder al formulario de login. 2. Ingresar usuario y contraseña inválidos. 3. Presionar "Entrar".	
Resultado esperado: - Página login.html se vuelve a renderizar. - Modal de error visible indicando credenciales inválidas. - No se crea sesión activa	
Evaluación de la Prueba: - El sistema respondió conforme a lo esperado.	



UNEAC

Iniciar Sesión

Entrar

Figura A- 1 Autenticación de usuarios

 **Comité Provincial UNEAC - Santiago de Cuba**

Bienvenido, admin [Cerrar sesión](#)

Inicio

Asociado

Calendario eventos

Cotización

Reportes

Gráficas

Secretaría de Filiales

Selecciona una opción en el menú lateral para comenzar.

Total de Asociados
2
"Construyendo cultura juntos"
[...Más](#)

Cumpleaños Mes
2
Asociados que celebran este mes
[...Más](#)

Eventos del Mes
2
Celebraciones que nos convocan
[...Más](#)

Cumpleaños de la Semana
Zolanch García Melian
(cumple)
27/12 32 años

Próximos Eventos
No hay eventos próximos programados

Figura A- 2 Página principal Dashboard

77

Comité Provincial UNEAC - Santiago de Cuba

Bienvenido, admin
 Cerrar sesión

Inicio

 Asociado

 Calendario eventos

 Cotización

 Reportes

 Gráficas

Gestión de Asociados

+ Nuevo Asociado

Buscar por nombre, apellidos, carnet, pasaporte o email

Buscar
 Limpiar

Lista de Asociados

2

Nombre Completo	Carnet	Teléfono	Email	Filial	Acciones
Zolanch García Melian Dec 27, 1993	93072722505	+5322659536 +5358328654	Sin email	Música	Editar Eliminar
Hirsel Paredes Vega Dec 19, 1987	87091633607	Sin teléfono	Sin email	Música	Editar Eliminar

Mostrando 1-2 de 2 asociados

1

Figura A- 3 Página de Gestión de Asociados

Comité Provincial UNEAC - Santiago de Cuba

Bienvenido, admin
 Cerrar sesión

Inicio

 Asociado

 Calendario eventos

 Cotización

 Reportes

 Gráficas

Crear Asociado

Datos Personales
 Membresía

Información Personal

Nombre *

 Segundo Nombre

 Apellido Paterno *

 Apellido Materno

 Fecha de Nacimiento *

 dd/mm/aaaa

 Carnet de Identidad *

11 dígitos sin espacios

 Número de Pasaporte

7 dígitos sin espacios

Contacto

Dirección Particular

 Correo Electrónico

 Teléfonos

Máximo 2 teléfonos por asociado

☐ Eliminar

Agregar Teléfono

Figura A- 4 Formulario de Gestión de Asociados



Figura A- 7 Página de Reportes de Asociados



Figura A- 8 Página Gráfica de Distribución de Asociados por filial

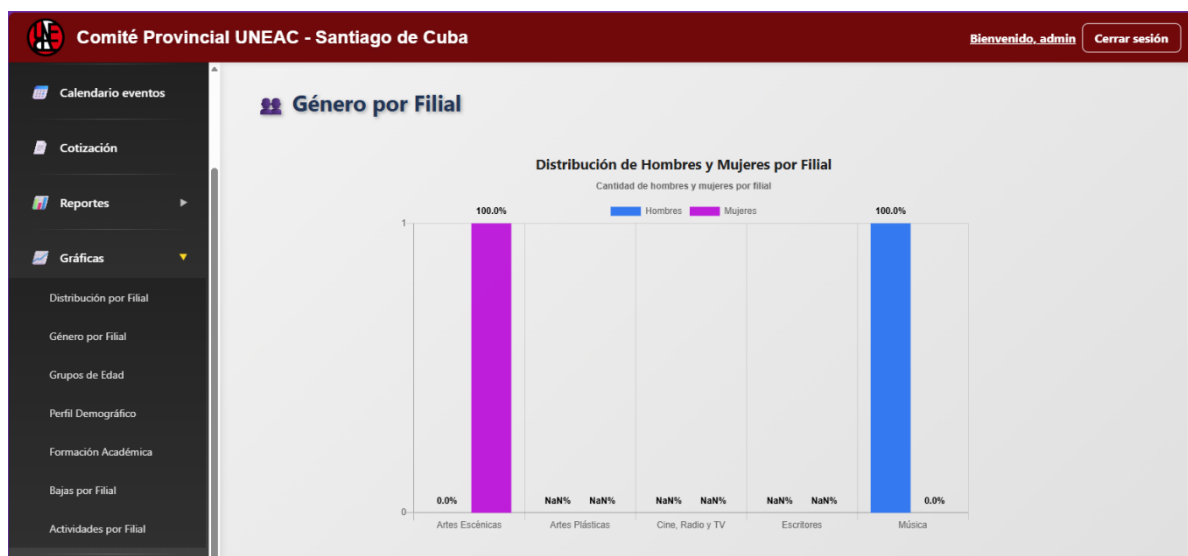


Figura A- 9 Página Gráfica de Distribución de Géneros de los Asociados por filial

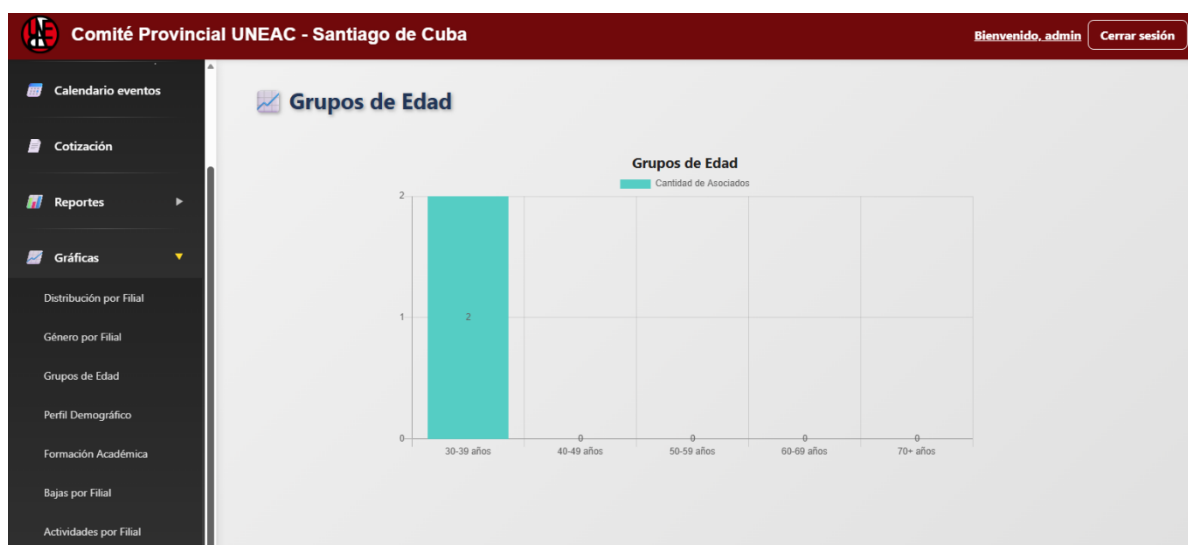


Figura A- 10 Página Gráfica de Distribución de Asociados por grupos de edad

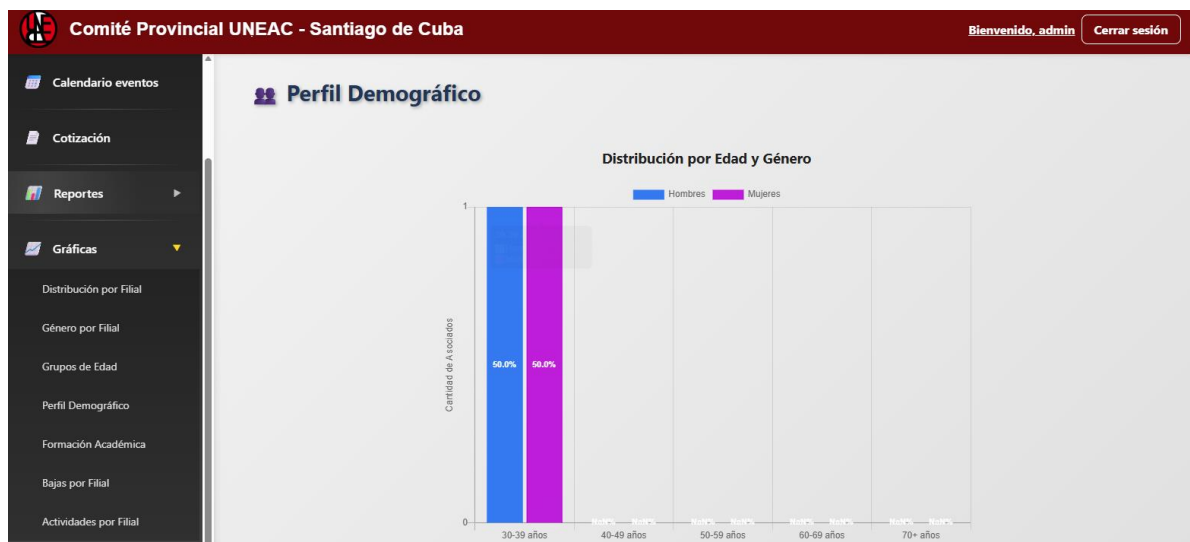


Figura A- 11 Página Gráfica de Distribución de Asociados por género y edades